

Artificial Intelligence

Capstone Project Final Report

©2023 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

HỆ THỐNG NHẬN DIỆN BIỂN BÁO GIAO THÔNG ỨNG DỤNG MÔ HÌNH YOLOv8

07/08/2025

Nhóm 14

**Vũ Minh Đức
Đặng Hoàng An
Trần Trọng Hoàn**

Mục lục

1. Giới thiệu chung

- 1.1. Giới thiệu dự án
- 1.2. Động lực và mục tiêu
- 1.3. Thành viên và phân công công việc
- 1.4. Lịch trình và mốc thời gian

2. Triển khai dự án

- 2.1. Ý tưởng của dự án
- 2.2. Thu thập dữ liệu
- 2.3. Xây dựng mã nguồn

3. Kết quả

- 3.1. Số liệu và biểu đồ
- 3.2. Thử nghiệm hệ thống

4. Kết luận

- 4.1. Kết luận chung
- 4.2. Hạn chế
- 4.3. Tiềm năng phát triển

5. Phụ lục và tài liệu tham khảo

6. Nhận xét và đánh giá của thành viên

7. Nhận xét và đánh giá của giảng viên

1. Giới thiệu chung

1.1. Giới thiệu dự án

Tình trạng giao thông hiện nay là vấn đề rất phức tạp, đặc biệt là trong các khu đô thị, các thành phố lớn. Việc đảm bảo an toàn cho người tham gia giao thông và nâng cao nhận thức giúp họ đã trở thành một trong những ưu tiên hàng đầu. Một trong những yếu tố quan trọng góp phần nâng cao ý thức và sự tuân thủ luật lệ giao thông là hệ thống biển báo giao thông đường bộ. Tuy nhiên, trong thực tế, không phải lúc nào người lái xe cũng có thể phát hiện và xử lý kịp thời các biển báo do nhiều yếu tố như tốc độ di chuyển cao, thời tiết xấu, tầm nhìn hạn chế hoặc thiếu tập trung.

Dự án hướng tới việc xây dựng một hệ thống nhận diện biển báo giao thông Việt Nam tự động sử dụng mô hình YOLOv8-Medium của Ultralytics – một trong những mô hình tiên tiến trong lĩnh vực thị giác máy tính. Hệ thống có khả năng phát hiện và phân loại các loại biển báo giao thông một cách nhanh chóng và chính xác trên hình ảnh hoặc video theo thời gian thực, từ đó có thể được tích hợp vào các thiết bị hỗ trợ người lái như camera hành trình, hệ thống lái tự động, hoặc ứng dụng di động.

1.2. Động lực và mục tiêu

1.2.1. Động lực

Trong thời đại công nghệ số, việc áp dụng trí tuệ nhân tạo (AI) và học sâu (Deep Learning) vào lĩnh vực giao thông đang trở thành xu hướng tất yếu nhằm nâng cao hiệu quả quản lý và đảm bảo an toàn cho người tham gia giao thông. Đặc biệt, việc tự động nhận diện biển báo giao thông không chỉ góp phần hỗ trợ người lái xe phản ứng kịp thời với các tình huống trên đường, mà còn là bước quan trọng trong quá trình phát triển các hệ thống xe tự hành, giám sát giao thông thông minh và thành phố thông minh (Smart City).

Tại Việt Nam, hiện nay chưa có nhiều hệ thống nhận diện biển báo được thiết kế riêng cho bộ quy chuẩn QCVN 41:2024/BGTVT, khiến cho việc áp dụng các công nghệ nước ngoài gặp nhiều hạn chế do sự khác biệt về thiết kế biển báo, ngôn ngữ và hệ thống luật ban hành. Do đó, việc nghiên cứu và xây dựng một mô hình nhận diện biển báo giao thông Việt Nam việc làm cần thiết và mang tính thực tiễn cao.

1.2.2. Mục tiêu

Dự án hướng tới các mục tiêu cụ thể như sau:

- Xây dựng tập dữ liệu huấn luyện mô hình bao gồm các hình ảnh biển báo giao thông tại Việt Nam, được gán nhãn đầy đủ và chính xác.
- Ứng dụng mô hình YOLOv8-Medium để xây dựng hệ thống có khả năng nhận diện và phân loại các loại biển báo theo thời gian thực.
- Cải tiến và mở rộng mô hình để ứng dụng vào các hệ thống thực tế như camera hành trình, hệ thống hỗ trợ người lái (ADAS), ứng dụng di động. Và có thể tích hợp mô hình xử lý ngôn ngữ tự nhiên (NLP).

1.3. Thành viên và phân công công việc

- **Vũ Minh Đức:** Lên ý tưởng, vẽ lộ trình triển khai dự án, hỗ trợ tìm kiếm tài liệu và thông tin; Quản lý dự án và xây dựng tài liệu.
- **Đặng Hoàng An:** Thu thập dữ liệu cho Dataset, xây dựng tài liệu và phát triển dự án, Kiểm thử hệ thống
- **Trần Trọng Hoàn:** Thu thập dữ liệu cho Dataset, gán nhãn cho dữ liệu, thiết kế và phát triển dự án, tối ưu hệ thống.

1.4. Lịch trình và mốc thời gian

Tuần 1 (21/6 - 28/6/2025)

- **Lên ý tưởng cho dự án**
 - **Nhiệm vụ:**
 - Lên ý tưởng lõi cho dự án.
 - Tham khảo ý tưởng trên các diễn đàn công nghệ, mạng xã hội, người có kinh nghiệm.
 - Đánh giá độ khả thi và tính thực tiễn của dự án.
 - **Chịu trách nhiệm:** Vũ Minh Đức, Đặng Hoàng An, Trần Trọng Hoàn.
 - **Kết quả:** Thống nhất được ý tưởng lõi cho dự án.

Tuần 2 (29/6 - 6/7/2025)

- **Thu thập dữ liệu cho Dataset và xử lý dữ liệu thu được**
 - **Nhiệm vụ:**
 - Thu thập dữ liệu cho Dataset cho việc huấn luyện mô hình.
 - Gán nhãn dữ liệu cho Dataset.
 - **Chịu trách nhiệm:** Đặng Hoàng An, Trần Trọng Hoàn

- **Kết quả:** Có được bộ dữ liệu biển báo giao thông Việt Nam gồm 12 lớp biển báo với tổng cộng 385 ảnh.

Tuần 3 và 4 (7/7 – 21/7/2025)

- **Xây dựng hệ thống**
 - **Nhiệm vụ:**
 - Xây dựng hệ thống nhận diện biển báo giao thông Việt Nam dựa trên mô hình YOLOv8-Medium với bộ dữ liệu đã thu thập.
 - Thực hiện quy trình Phát Triển – Kiểm Thử đan xen để đánh giá hiệu quả và hiệu suất của hệ thống.
 - Tối ưu hóa hệ thống sao cho hệ thống đạt hiệu suất tốt nhất có thể.
 - **Chịu trách nhiệm:** Đặng Hoàng An, Trần Trọng Hoàn, Vũ Minh Đức (hỗ trợ).
 - **Kết quả:** Xây dựng tương đối thành công hệ thống.

Tuần 5 (21/7 - 28/7/2025)

- **Thử nghiệm và xây dựng tài liệu**
 - **Nhiệm vụ:**
 - Thử nghiệm hệ thống để kiểm tra hiệu suất, độ hiệu quả.
 - Xây dựng tài liệu về hệ thống.
 - Sửa chữa, tối ưu lại mã nguồn nếu có vấn đề phát sinh trong tầm kiểm soát.
 - **Chịu trách nhiệm:** Đặng Hoàng An, Trần Trọng Hoàn, Vũ Minh Đức.
 - **Kết quả:** Hoàn tất tài liệu về hệ thống và tối ưu tương đối hiệu suất cho hệ thống.

Ghi chú:

- Do thời gian và kiến thức có hạn nên dự án của nhóm mang tính nghiên cứu nhiều hơn là phát triển với mục đích thương mại và ứng dụng vào thực tế. Nên các nhiệm vụ ưu tiên tập trung vào các hoạt động cốt lõi như: Thu thập dữ liệu, xử lý dữ liệu, xây dựng một hệ thống cơ bản, xây dựng tài liệu báo cáo.
- Thảo luận nhóm được tổ chức vào thứ 7 hàng tuần để đánh giá hiệu suất và tiến độ của dự án. Vũ Minh Đức là người chủ trì buổi thảo luận.

2. Triển khai dự án

2.1. Ý tưởng của dự án

2.1.1. Ý tưởng lõi

Ý tưởng cốt lõi của dự án là xây dựng một hệ thống có khả năng nhận diện chính xác các biển báo giao thông Việt Nam dựa trên hình ảnh đầu vào từ camera hoặc video. Dự án ứng dụng mô hình học sâu YOLOv8m – một trong những mô hình hiện đại và hiệu quả nhất trong lĩnh vực thị giác máy tính để thực hiện việc phát hiện và phân loại biển báo theo thời gian thực.

Tính năng cốt lõi mà hệ thống hướng tới bao gồm:

- Phát hiện nhanh và chính xác các loại biển báo giao thông đường bộ Việt Nam theo chuẩn QCVN 41:2024/BGTVT.
- Phân loại chính xác biển báo như: biển cấm đỗ xe, biển nguy hiểm giao nhau với đường không ưu tiên, biển hiệu lệnh, biển chỉ dẫn,...
- Xử lý thời gian thực, phù hợp tích hợp vào các hệ thống như camera hành trình, xe tự lái, ứng dụng giao thông,...
- Độ chính xác cao trong nhiều điều kiện: Mờ, lóa, chiếu bóng, ngược sáng,...

2.1.2. Tính năng mở rộng

Bên cạnh tính năng nhận diện hình ảnh, dự án dự kiến sẽ mở rộng tích hợp thêm mô hình xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP) để giải quyết các trường hợp biển báo có chứa văn bản mang thông tin địa phương, tên đường, khu vực cụ thể.

Ví dụ: Biển chỉ dẫn "Rẽ trái 2km đến Thịnh Đán", "Đi thẳng 4km đến Bến xe Thái Nguyên",... thường chứa nội dung dạng chữ viết không cố định theo quy chuẩn. Việc tích hợp NLP sẽ giúp trích xuất và hiểu nội dung chữ viết trên biển báo, giúp hệ thống đọc hiểu thông tin ngữ nghĩa và phục vụ các ứng dụng nâng cao như hướng dẫn đường đi, cảnh báo khu vực đặc biệt,...

2.2. Thu thập dữ liệu

Trong khuôn khổ dự án này, việc xây dựng một tập dữ liệu chất lượng cao và chính xác theo quy chuẩn QCVN 41:2024/BGTVT là một bước quan trọng thể thiếu trong quy trình xây dựng hệ thống.

2.2.1. Nguồn dữ liệu

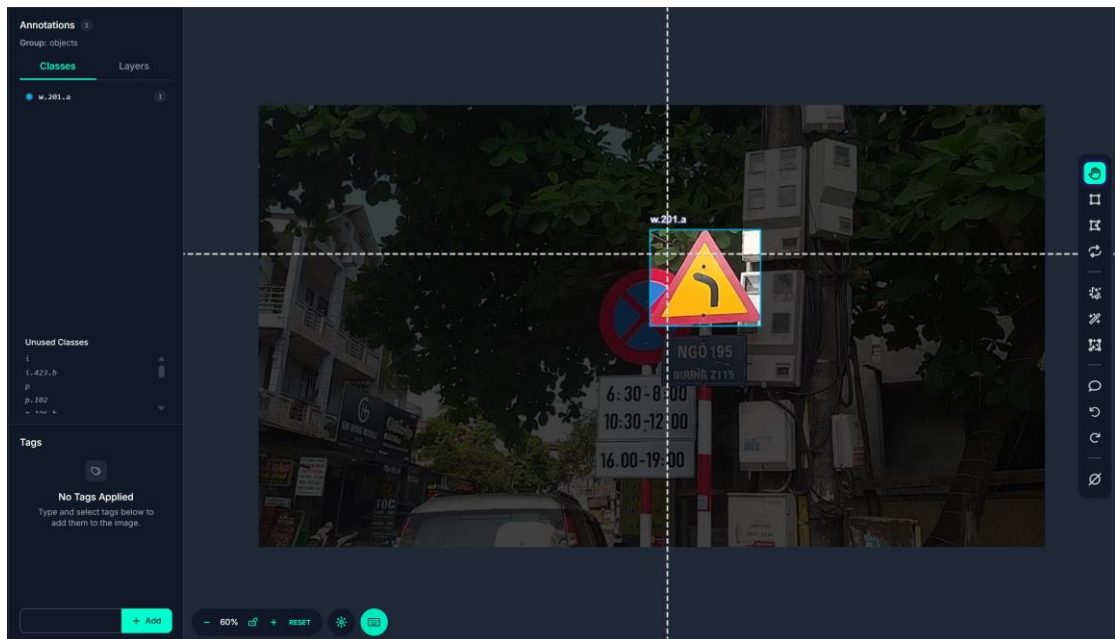
Ban đầu nhóm dự định lấy dữ liệu từ các nguồn công khai miễn phí như Kaggle. Nhưng sau khi xem xét kỹ lưỡng nhóm, nhóm quyết định sẽ đi thu thập dữ liệu trực tiếp bằng cách quay video và chụp ảnh trên các tuyến đường chính trong khu vực Thành phố Thái Nguyên. Và kết quả chúng tôi đã thu được bộ dữ liệu bao gồm 12 lớp biển với tổng cộng 385 ảnh được chụp với nhiều góc độ chiếu sáng và điều kiện khác nhau.

2.2.2. Tiền xử lý và gán nhãn dữ liệu

Sau khi có được bộ dữ liệu, dữ liệu cần được xử lý nhằm chuẩn hóa đầu vào cho mô hình. Chúng tôi đã đẩy bộ dữ liệu lên Roboflow – nền tảng cung cấp các công cụ hỗ trợ rất tốt trong lĩnh vực thị giác máy tính để tiến hành gán nhãn dữ liệu. Tổng cộng chúng tôi có 12 lớp nhãn chia thành 4 nhóm biển báo:

- Biển báo chỉ dẫn: *r.308.b*
- Biển báo cấm: *p.102 / p.106.b / p.130 / p.131.a*
- Biển cảnh báo nguy hiểm: *w.201.a / w.203.c / w.207.b / w.207.c / w.209*
- Biển hiệu lệnh: *i.423.b*
- Biển báo phụ và biển báo chỉ dẫn có chữ: Chúng tôi để nhãn là “*sus*”

```
names: ['i.423.b', 'p.102', 'p.106.b', 'p.130',  
'p.131.a', 'r.308.b', 'sus', 'w.201.a', 'w.203.c',  
'w.207.b', 'w.207.c', 'w.209']
```

Sau khi gắn nhãn cho bộ dữ liệu, chúng tôi tiến hành tăng cường dữ liệu (Data Augmentation) bằng các kỹ thuật như xoay, thay đổi độ sáng, độ mờ, độ nhiễu, độ tương phản, cường độ màu, cường độ thu phóng,... để mô phỏng các điều kiện ánh sáng và thời tiết khác nhau.

Augmentations

Outputs per training example: 3

Crop: 0% Minimum Zoom, 20% Maximum Zoom

Rotation: Between -15° and $+15^{\circ}$

Grayscale: Apply to 5% of images

Hue: Between -15° and $+15^{\circ}$

Saturation: Between -25% and +25%

Brightness: Between -15% and +15%

Exposure: Between -10% and +10%

Blur: Up to 1px

Noise: Up to 0.26% of pixels

2.2.3. Phân chia dữ liệu

Tập dữ liệu sau khi được gán nhãn sẽ được chia thành 3 phần:

- **Tập huấn luyện (*Training set*):** chiếm khoảng 88% dữ liệu.
- **Tập kiểm tra (*Validation set*):** chiếm khoảng 8%, dùng để điều chỉnh mô hình.
- **Tập kiểm thử (*Test set*):** chiếm khoảng 4%, dùng để đánh giá khách quan hiệu suất của mô hình.

Việc phân chia đảm bảo không có trùng lặp giữa các tập dữ liệu nhằm tránh hiện tượng overfitting và đảm bảo tính khách quan trong quá trình đánh giá.

2.3. Xây dựng mã nguồn

2.3.1. Cấu trúc mã nguồn dự án

```
Traffic-Sign-Detection/
├── all_weight/                # Thư mục trọng số đã huấn luyện
├── dataset/                  # Dữ liệu gốc và cấu hình YOLO
│   ├── train/               # Tập dữ liệu huấn luyện
│   ├── valid/               # Tập dữ liệu kiểm tra
│   ├── test/                # Tập dữ liệu kiểm thử
│   └── data.yaml            # File cấu hình nhãn
├── input/                   # Ảnh, video đầu vào
├── output/                  # Kết quả xuất ra (ảnh, Log,...)
├── real_time_output/        # Kết quả video
├── runs/                    # Kết quả huấn luyện, Logs,...
├── src/                     # Mã nguồn phụ trợ (nếu có)
├── test_output/             # Kết quả kiểm thử
├── config.py                # File cấu hình chung
├── requirements.txt          # Thư viện cần cài đặt
├── README.md                # Hướng dẫn tổng quan dự án
├── train.py                 # Script huấn luyện YOLO
├── predict.py               # Script dự đoán ảnh/video
├── real_time_predict_nlp_hybrid.py # Nhận diện biển báo
├── utils.py                 # Hàm tiện ích
└── ...                      # Các file báo cáo, kiểm thử khác
```

2.3.2. Cấu hình thông số cho việc huấn luyện

Toàn bộ các tham số phục vụ cho quá trình huấn luyện, kiểm thử và dự đoán được quản lý tập trung trong file **config.py**. Và dưới đây là thông số huấn luyện cho mô hình:

- **LEARNING_RATE = 0.01**: Tốc độ học, quyết định mức độ điều chỉnh trọng số sau mỗi lần cập nhật. Giá trị lớn giúp mô hình học nhanh hơn nhưng dễ bị dao động, giá trị nhỏ giúp học ổn định hơn nhưng chậm.
- **WEIGHT_DECAY = 0.0005**: Tham số regularization (giảm thiểu overfitting), giúp phạt các trọng số lớn trong mô hình, từ đó mô hình tổng quát tốt hơn.
- **MOMENTUM = 0.937**: Tham số động lượng, giúp quá trình tối ưu hóa “mượt” hơn bằng cách kết hợp tốc độ cập nhật hiện tại với tốc độ trước đó, giúp tránh kẹt ở các điểm cực tiểu cục bộ.

2.3.3. Huấn luyện mô hình

Quy trình huấn luyện, đánh giá và kiểm thử mô hình được thực hiện tự động hóa trong file **train.py** với các chức năng chính:

- **Khởi tạo huấn luyện**: Lớp TrafficSignTrainer chịu trách nhiệm toàn bộ quy trình training. Hàm `__init__` khởi tạo cấu hình, augmentation và tạo thư mục lưu trọng số.
- **Thiết lập môi trường**: Hàm `setup_training()` tạo các thư mục cần thiết, sinh file **data.yaml** nếu chưa có, và in ra thông số cấu hình.

```
68
69     def setup_training(self):
70         """Setup training environment and create necessary files/directories"""
71         # Create necessary directories
72         self.config.create_directories()
73
74         # Generate dataset.yaml only if it doesn't exist
75         self.data_yaml_path = 'data.yaml'
76         if not os.path.exists(self.data_yaml_path):
77             dataset_content = self.config.get_dataset_yaml()
78             with open(self.data_yaml_path, 'w', encoding='utf-8') as f:
79                 f.write(dataset_content)
80             print(f"Created new data.yaml file at: {self.data_yaml_path}")
81         else:
82             print(f"Using existing data.yaml file at: {self.data_yaml_path}")
83
84         # Print training configuration
85         print("\nTraining Configuration:")
86         print(f"Model: {self.config.MODEL_SIZE}")
87         print(f"Image Size: {self.config.IMAGE_SIZE}")
88         print(f"Batch Size: {self.config.BATCH_SIZE}")
89         print(f"Epochs: {self.config.EPOCHS}")
90         print(f"Learning Rate: {self.config.LEARNING_RATE}")
91         print(f"Device: {torch.device('cuda' if torch.cuda.is_available() else 'cpu')}")
92
```

- **Huấn luyện mô hình:** Hàm `train()` thực hiện huấn luyện mô hình YOLOv8 với các tham số đã cấu hình, lưu lại trọng số tốt nhất và cuối cùng vào thư mục riêng biệt cho từng lần train. Kết quả huấn luyện (mAP50, mAP50-95) được in ra màn hình.

```

92
93 def train(self):
94     """Train the YOLOv8 model on traffic sign dataset"""
95     try:
96         # Load the model
97         model = YOLO(self.config.MODEL_SIZE)
98
99         # Train the model
100         results = model.train(
101             data=self.data_yaml_path,
102             epochs=self.config.EPOCHS,
103             imgsz=self.config.IMAGE_SIZE,
104             batch=self.config.BATCH_SIZE,
105             name='traffic_sign_detection',
106             patience=50, # Early stopping patience
107             save=True, # Save best and last checkpoints
108             device=0 if torch.cuda.is_available() else 'cpu',
109             verbose=True,
110
111             # Optimizer parameters
112             lr=self.config.LEARNING_RATE,
113             weight_decay=self.config.WEIGHT_DECAY,
114             momentum=self.config.MOMENTUM,
115
116             # Augmentation parameters
117             flipud=self.config.VERTICAL_FLIP,
118            fliplr=self.config.HORIZONTAL_FLIP,
119             mosaic=0.5, # Mosaic augmentation
120             mixup=0.3, # Mixup augmentation
121             degrees=self.config.ROTATION,
122
123             # Save best model
124             save_period=10, # Save checkpoint every 10 epochs
125             project='runs', # Project name
126             exist_ok=True, # Overwrite existing experiment
127
128             # Additional parameters for better convergence
129             warmup_epochs=3.0, # Warmup epochs
130             warmup_momentum=0.8, # Warmup momentum
131             warmup_bias_lr=0.1, # Warmup initial bias lr
132             box=7.5, # Box loss gain
133             cls=0.5, # Classification loss gain
134             dfl=1.5, # DFL loss gain
135             close_mosaic=10, # Close mosaic augmentation for last 10 epochs
136             workers=self.config.WORKERS,
137         )
138
139         # Lưu best.pt và last.pt vào thư mục all_weight/trainX
140         best_model_path = os.path.join('runs', 'traffic_sign_detection', 'weights', 'best.pt')
141         last_model_path = os.path.join('runs', 'traffic_sign_detection', 'weights', 'last.pt')
142         train_dir = self.get_next_train_dir()
143         os.makedirs(train_dir, exist_ok=True)
144         if os.path.exists(best_model_path):
145             dest_best = os.path.join(train_dir, 'best.pt')
146             os.replace(best_model_path, dest_best)
147             print(f"\nBest model saved to: {dest_best}")
148
149         if os.path.exists(last_model_path):
150             dest_last = os.path.join(train_dir, 'last.pt')
151             os.replace(last_model_path, dest_last)
152             print(f"\nLast model saved to: {dest_last}")
153
154         # Print training results
155         print("\nTraining Results:")
156         print(f"Best mAP50: {results.results_dict.get('metrics/mAP50(B)', 0):.4f}")
157         print(f"Best mAP50-95: {results.results_dict.get('metrics/mAP50-95(B)', 0):.4f}")
158
159         return results
160
161     except Exception as e:
162         print(f"\nError during training: {str(e)}")
163         raise
164

```

- **Đánh giá trên tập validation:** Hàm `validate()` đánh giá mô hình trên tập validation, in ra các chỉ số mAP.

```
164
165 def validate(self, model_path: str = None):
166     """
167     Validate the trained model
168
169     Args:
170         model_path: Path to the model to validate (default: best model)
171     """
172     if model_path is None:
173         model_path = self.config.BEST_MODEL_PATH
174
175     if not os.path.exists(model_path):
176         print(f"Model not found at: {model_path}")
177         return
178
179     try:
180         # Load the model
181         model = YOLO(model_path)
182
183         # Validate
184         results = model.val(
185             data=self.data_yaml_path,
186             imgsz=self.config.IMAGE_SIZE,
187             batch=self.config.BATCH_SIZE,
188             device=0 if torch.cuda.is_available() else 'cpu',
189             verbose=True
190         )
191
192         # Print validation results
193         print("\nValidation Results:")
194         print(f"mAP50: {results.results_dict.get('metrics/mAP50(B)', 0):.4f}")
195         print(f"mAP50-95: {results.results_dict.get('metrics/mAP50-95(B)', 0):.4f}")
196
197         return results
198
199     except Exception as e:
200         print(f"\nError during validation: {str(e)}")
201         raise
202
```

- **Đánh giá trên tập test:** Hàm `test()` tự động đánh giá mô hình trên tập test, sử dụng đúng cấu hình và in ra kết quả.

```

17 def test(self, model_path: str = None):
18     """
19     Test the trained model on the test set
20     Args:
21     | model_path: Path to the model to test (default: best model)
22     """
23     if model_path is None:
24         model_path = self.config.BEST_MODEL_PATH
25
26     if not os.path.exists(model_path):
27         print(f"Model not found at: {model_path}")
28         return
29
30     try:
31         # Load the model
32         model = YOLO(model_path)
33
34         # Test (use split='test' if supported
35         # else set data to self.data_yaml_path and YOLOv8 will use test set if defined)
36         results = model.val(
37             data=self.data_yaml_path,
38             imgsz=self.config.IMAGE_SIZE,
39             batch=self.config.BATCH_SIZE,
40             device=0 if torch.cuda.is_available() else 'cpu',
41             split='test',
42             verbose=True
43         )
44
45         # Print test results
46         print("\nTest Results:")
47         print(f"mAP50: {results.results_dict.get('metrics/mAP50(B)', 0):.4f}")
48         print(f"mAP50-95: {results.results_dict.get('metrics/mAP50-95(B)', 0):.4f}")
49
50         return results
51
52     except Exception as e:
53         print(f"\nError during test: {str(e)}")
54         raise

```

- **Chạy toàn bộ quy trình:** Hàm `main()` lần lượt gọi các bước: setup, train, validate, test. Khi chạy file ***train.py***, toàn bộ quy trình sẽ được thực hiện tự động.

```

204 def main():
205     """Main function to run training"""
206     trainer = TrafficSignTrainer()
207
208     print("Setting up training...")
209     trainer.setup_training()
210
211     print("\nStarting training...")
212     results = trainer.train()
213
214     print("\nStarting validation...")
215     val_results = trainer.validate()
216
217     print("\nStarting test on test set...")
218     test_results = trainer.test()
219     print("\nTraining, validation and test completed!")
220
221 if __name__ == "__main__":
222     main()

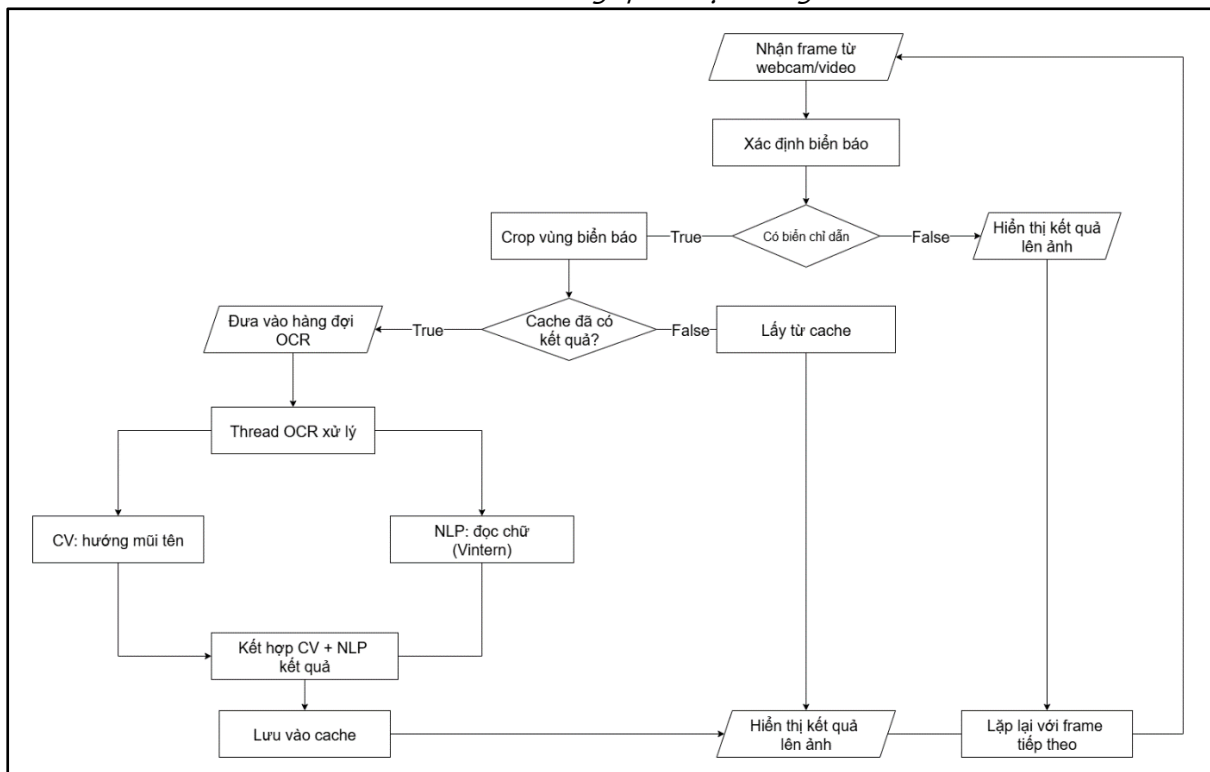
```

2.3.4. Xây dựng hệ thống nhận diện

Hệ thống nhận diện biển báo giao thông sẽ được xây dựng để có thể hoạt động theo thời gian thực. Hệ thống sẽ kết hợp sử dụng mô hình YOLOv8 để nhận diện biển báo và mô hình xử lý ngôn ngữ tự nhiên (NLP) để nhận diện sơ lược nội dung được ghi trên biển báo. Ngoài ra hệ thống sẽ sử dụng các thuật toán xử lý ảnh như: So khớp mẫu (Template Matching), Phân tích đường viền (Contour Analysis), Phân tích hướng cạnh (Edge Direction Analysis), Bỏ phiếu trọng số (Weighted Voting/Fusion) để nhận diện *hướng mũi trên chỉ dẫn trên biển báo*.

A [Nhận frame từ webcam/video] --> B [Phát hiện biển báo bằng YOLOv8]
 B --> C {Có biển chỉ dẫn?}
 C -- Không --> D [Hiển thị kết quả]
 C -- Có --> E [Crop vùng biển báo nghi ngờ]
 E --> F {Cache đã có kết quả?}
 F -- Có --> G [Lấy kết quả từ cache]
 F -- Không --> H [Đưa vào hàng đợi OCR]
 H --> I [Thread OCR xử lý]
 I --> J1 [Nhận diện hướng mũi tên (CV)]
 I --> J2 [Đọc chữ bằng Vintern NLP]
 J1 & J2 --> K [Kết hợp kết quả CV + NLP]
 K --> L [Lưu vào cache]
 G & L --> M [Hiển thị kết quả lên ảnh/video]
 M --> N [Lặp lại với frame tiếp theo]

< Workflow tổng quát hệ thống >



2.3.4.a. Xác định biển báo

a. Khởi tạo mô hình

Ở phần khởi tạo class `RealTimeTrafficSignDetectorNLPHybrid`, mô hình YOLOv8 được load từ file trọng số (`best.pt`) đã huấn luyện:

b. Dự đoán trên từng frame

Khi nhận một frame ảnh từ webcam hoặc video, hàm `predict_frame` sẽ:

- Tiền xử lý ảnh (tăng cường chất lượng nếu cần).
- Gọi mô hình YOLOv8 để dự đoán:

```
results = self.model.predict(
    enhanced,
    conf=self.config.CONFIDENCE_THRESHOLD,
    iou=self.config.NMS_THRESHOLD,
    max_det=self.config.MAX_DETECTIONS,
    verbose=False
)
```

- Kết quả trả về là danh sách các bounding box, mỗi box bao gồm:
 - Tọa độ (x1, y1, x2, y2)
 - Độ tin cậy (confidence)
 - Loại biển báo (class index)

c. Xử lý kết quả

Kết quả được chuyển thành các dictionary chứa thông tin:

- `bbox`: vị trí biển báo
- `confidence`: độ tin cậy
- `class_id`, `class_label`: loại biển báo

2.3.4.b. Xác định hướng mũi tên trên biển báo

a. Mục đích

Trong các lớp biển báo giao thông đường bộ, có một số lớp biển báo phụ hoặc biển báo chỉ dẫn sẽ ghi thêm thông tin dạng **chữ** trên đó. Vậy nên

để xác định chính xác chỉ dẫn trên biển, chúng tôi đã xây dựng một hệ thống có khả năng đọc thông tin định dạng chữ trên biển báo và xác định hướng mũi tên chỉ dẫn trên biển báo để có thể thông báo chính xác cho người sử dụng.



Và ArrowDetector là module sử dụng các thuật toán Thị giác máy tính (Computer Vision) truyền thống để xác định hướng mũi tên trên biển báo chỉ dẫn theo các hướng: Trái, Phải, Đi thẳng,...

b. Các thuật toán

- Template Matching (So khớp mẫu)
 - Tạo ra nhiều mẫu mũi tên (trái, phải, thẳng) với các kích thước và góc xoay khác nhau.

```
189
190 def _create_arrow_templates(self):
191     """Tạo templates mũi tên cho các hướng với rotation để handle góc xiên"""
192     templates = {}
193
194     # Template cho mũi tên trái (pointing left) - nhiều kích thước + rotation
195     for size in [20, 30, 40, 50]:
196         for angle in [0, -15, 15]: # Thêm template nghiêng ±15 độ
197             left_arrow = np.zeros((size, size), dtype=np.uint8)
198             # Vẽ mũi tên trái: <
199             pts = np.array([[size*0.7, size*0.3], [size*0.3, size*0.5], [size*0.7, size*0.7]], dtype=np.int32)
200             cv2.fillPoly(left_arrow, [pts], 255)
201
202             # Rotate template nếu có angle
203             if angle != 0:
204                 M = cv2.getRotationMatrix2D((size//2, size//2), angle, 1.0)
205                 left_arrow = cv2.warpAffine(left_arrow, M, (size, size))
206
207             templates[f'left_{size}_{angle}'] = left_arrow
208
209     # Template cho mũi tên phải (pointing right) - nhiều kích thước + rotation
210     for size in [20, 30, 40, 50]:
211         for angle in [0, -15, 15]: # Thêm template nghiêng ±15 độ
212             right_arrow = np.zeros((size, size), dtype=np.uint8)
213             # Vẽ mũi tên phải: >
214             pts = np.array([[size*0.3, size*0.3], [size*0.7, size*0.5], [size*0.3, size*0.7]], dtype=np.int32)
215             cv2.fillPoly(right_arrow, [pts], 255)
216
217             # Rotate template nếu có angle
218             if angle != 0:
219                 M = cv2.getRotationMatrix2D((size//2, size//2), angle, 1.0)
220                 right_arrow = cv2.warpAffine(right_arrow, M, (size, size))
221
222             templates[f'right_{size}_{angle}'] = right_arrow
223
224     # Template cho mũi tên thẳng (pointing up) - nhiều kích thước + rotation
225     for size in [20, 30, 40, 50]:
226         for angle in [0, -10, 10]: # Ít rotation hơn cho straight
227             up_arrow = np.zeros((size, size), dtype=np.uint8)
228             # Vẽ mũi tên lên: ^
229             pts = np.array([[size*0.3, size*0.7], [size*0.5, size*0.3], [size*0.7, size*0.7]], dtype=np.int32)
230             cv2.fillPoly(up_arrow, [pts], 255)
231
232             # Rotate template nếu có angle
233             if angle != 0:
234                 M = cv2.getRotationMatrix2D((size//2, size//2), angle, 1.0)
235                 up_arrow = cv2.warpAffine(up_arrow, M, (size, size))
236
237             templates[f'up_{size}_{angle}'] = up_arrow
238
239     return templates
240
```

- Tiền xử lý biến báo (nếu cần).

```
blurred = cv2.GaussianBlur(gray, (3, 3), 0)
edges = cv2.Canny(blurred, 50, 150)
kernel = np.ones((2,2), np.uint8)
edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4,4))
enhanced_gray = clahe.apply(gray)
```

- Dùng `cv2.matchTemplate` để so khớp từng mẫu với ảnh biển báo đã tiền xử lý (làm mịn, tăng tương phản, phát hiện biên).

```

670
671     for template_name, template in self.arrow_templates.items():
672         # Match với cả edge image và enhanced gray
673         result_edge = cv2.matchTemplate(edges, template, cv2.TM_CCOEFF_NORMED)
674         result_gray = cv2.matchTemplate(enhanced_gray, template, cv2.TM_CCOEFF_NORMED)
675
676         # Combine cả 2 kết quả
677         combined_result = 0.6 * result_edge + 0.4 * result_gray
678         _, max_val, _, max_loc = cv2.minMaxLoc(combined_result)
679
680         direction = template_name.split('_')[0]
681         size = int(template_name.split('_')[1])
682         angle = int(template_name.split('_')[2])
683
684         # Improved weight calculation
685         size_weight = 0.5 + (size - 20) / 60 # 0.5 to 1.0
686         angle_weight = 1.0 - abs(angle) / 20 # Penalty cho angle cao
687
688         # Quality boost cho high-confidence matches
689         quality_boost = 1.0
690         if max_val > 0.6:
691             quality_boost = 1.2
692         elif max_val > 0.4:
693             quality_boost = 1.1
694
695         weighted_score = max_val * size_weight * angle_weight * quality_boost
696
697         # Map to Vietnamese và store individual scores
698         vn_direction = "unknown"
699         if direction == "left":
700             vn_direction = "rẽ trái"
701         elif direction == "right":
702             vn_direction = "rẽ phải"
703         elif direction == "up":
704             vn_direction = "đi thẳng"
705
706         if vn_direction != "unknown":
707             direction_scores[vn_direction].append(weighted_score)
708

```

- Tính điểm số cho từng hướng dựa trên mức độ khớp, kích thước, góc xoay, và chất lượng khớp.

```

direction = template_name.split('_')[0]
size = int(template_name.split('_')[1])
angle = int(template_name.split('_')[2])
size_weight = 0.5 + (size - 20) / 60
angle_weight = 1.0 - abs(angle) / 20
quality_boost = 1.0
if max_val > 0.6:
    quality_boost = 1.2
elif max_val > 0.4:
    quality_boost = 1.1
weighted_score = max_val * size_weight * angle_weight * quality_boost

```

- Hỗ trợ nhận diện nhiều hướng cùng lúc (multi-direction).

```

708
709 # Aggregate scores với statistical analysis
710 final_direction_scores = {}
711 for direction, scores in direction_scores.items():
712     if scores:
713         # Use both max và average để balance outliers
714         max_score = max(scores)
715         avg_score = sum(scores) / len(scores)
716         # Weighted combination: 70% max, 30% average
717         final_score = 0.7 * max_score + 0.3 * avg_score
718         final_direction_scores[direction] = final_score
719
720 # Filter directions có confidence đủ cao với adaptive threshold
721 detected_directions = {}
722 total_score = sum(final_direction_scores.values())
723
724 if total_score > 0:
725     max_score = max(final_direction_scores.values())
726
727     for direction, score in final_direction_scores.items():
728         normalized_score = score / total_score
729
730         # Adaptive threshold dựa trên score distribution
731         if max_score > 0.6: # Có 1 direction rất dominant
732             threshold = 0.2
733         else: # Scores tương đương nhau
734             threshold = 0.15
735
736         if normalized_score > threshold and score > 0.1: # Absolute minimum
737             detected_directions[direction] = normalized_score
738

```

- Chuẩn hóa và lọc kết quả, fallback về hướng có điểm số cao nhất nếu không xác định được gì.

```

738
739 # Fallback nếu không detect được gì
740 if not detected_directions and final_direction_scores:
741     best_dir = max(final_direction_scores, key=final_dir_score.get)
742     best_score = final_direction_scores[best_dir] / max(total_score, 1e-6)
743     if best_score > 0.05: # Very low threshold cho fallback
744         detected_directions[best_dir] = best_score
745
746 overall_confidence = min(sum(detected_directions.values()) * 1.1, 1.0) # Slight boost
747 return detected_directions, overall_confidence
748

```

- Contour Analysis (Phân tích đường viền)
 - Chuyển ảnh sang nhị phân, tìm các contour (đường viền khép kín).

```

_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

```

- Duyệt qua từng contour, kiểm tra diện tích, số đỉnh, hình dạng.

```

759
760     for contour in contours:
761         area = cv2.contourArea(contour)
762         if area < 100: # Skip quá nhỏ
763             continue
764
765         # Approximate contour shape
766         epsilon = 0.02 * cv2.arcLength(contour, True)
767         approx = cv2.approxPolyDP(contour, epsilon, True)
768
769         if len(approx) >= 3: # Có ít nhất 3 điểm (triangle-like)
770             # Tìm tip (đỉnh nhọn) của mũi tên
771             direction, confidence = self._find_arrow_tip(approx)
772             if direction != "unknown" and confidence > 0.3:
773                 if direction not in direction_detections:
774                     direction_detections[direction] = []
775                     direction_detections[direction].append(confidence)
776

```

- Nếu contour đủ lớn và có hình dạng giống mũi tên (tam giác hoặc đa giác), sẽ tìm điểm tip (đầu nhọn) của mũi tên.

```

if len(approx) >= 3: # Có ít nhất 3 điểm (triangle-like)
    # Tìm tip (đỉnh nhọn) của mũi tên
    direction, confidence = self._find_arrow_tip(approx)
    if direction != "unknown" and confidence > 0.3:
        if direction not in direction_detections:
            direction_detections[direction] = []
            direction_detections[direction].append(confidence)

```

- Tính vector từ tâm contour đến tip, chuyển thành góc, từ đó xác định hướng (trái, phải, thẳng).

```

887
888 def _find_arrow_tip(self, approx_points) -> tuple:
889     """Tìm hướng tip của mũi tên từ các điểm contour với angle normalization cải thiện"""
890     try:
891         points = approx_points.reshape(-1, 2)
892         if len(points) < 3:
893             return "unknown", 0.0
894
895         # Tính center của shape
896         center = np.mean(points, axis=0)
897
898         # Tìm điểm xa nhất từ center (có thể là tip)
899         distances = [euclidean(point, center) for point in points]
900         tip_idx = np.argmax(distances)
901         tip_point = points[tip_idx]
902
903         # Xác định hướng từ center đến tip
904         direction_vector = tip_point - center
905         angle_rad = math.atan2(direction_vector[1], direction_vector[0])
906
907         # Normalize angle to [0, 360) degrees để tránh nhầm lẫn
908         angle_deg = (math.degrees(angle_rad) + 360) % 360
909
910         # Classify angle thành direction với boundary rõ ràng hơn
911         if 315 <= angle_deg or angle_deg < 45: # Right (0° ±45°)
912             return "rẽ phải", 0.75
913         elif 135 <= angle_deg < 225: # Left (180° ±45°)
914             return "rẽ trái", 0.75
915         elif 45 <= angle_deg < 135: # Down/Southeast (90° ±45°) -> đi thẳng
916             return "đi thẳng", 0.65
917         elif 225 <= angle_deg < 315: # Up/Northwest (270° ±45°) -> đi thẳng
918             return "đi thẳng", 0.70
919
920         return "unknown", 0.0
921
922     except Exception as e:
923         print(f"[ERROR] Find arrow tip: {e}")
924         return "unknown", 0.0
925

```

- Edge Direction Analysis
 - Dùng Sobel để tính gradient theo hai trục, xác định hướng các biên mạnh nhất.
 - Sử dụng Sobel để tính đạo hàm theo trục x (sobelx) và trục y (sobely).

```

# Tính gradients với kernel size lớn hơn để stable hơn
sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=5)
sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=5)

```


- Tính độ lớn (magnitude) và hướng (direction) của vector gradient tại mỗi pixel.

```
# Tính magnitude và direction
magnitude = np.sqrt(sobelx**2 + sobely**2)
direction = np.arctan2(sobely, sobelx)
```

- Tạo histogram hướng, tìm góc chiếm ưu thế nhất.
 - Chỉ xét các pixel có magnitude lớn (biên mạnh).

```
# Chỉ xét pixels có magnitude cao (edges mạnh) với adaptive threshold
magnitude_threshold = np.percentile(magnitude, 85)
strong_edges_mask = magnitude > magnitude_threshold
```

- Lấy hướng của các biên mạnh, chuyển sang $0^\circ - 360^\circ$.

```
strong_directions_deg = (np.degrees(strong_directions_rad) + 360) % 360
```

- Tạo histogram hướng, trọng số là độ lớn biên

```
hist, bins = np.histogram(strong_directions_deg, bins=72, range=(0, 360),
                           weights=strong_magnitudes)
```

- Làm mượt histogram để giảm nhiễu, tìm các đỉnh lớn nhất (dominant angles).

```
# Smooth histogram để giảm noise
from scipy import ndimage
hist_smooth = ndimage.gaussian_filter1d(hist, sigma=1.0)

# Tìm top 3 dominant directions để cross-validation
top_bins = np.argsort(hist_smooth)[-3:]
dominant_angles = [(bins[i] + bins[i + 1]) / 2 for i in top_bins]
dominant_counts = [hist_smooth[i] for i in top_bins]
```

- Chuyển góc này thành hướng giao thông (trái, phải, thẳng) với các ngưỡng rõ ràng.
 - Lấy góc chiếm ưu thế nhất (**primary_angle**), so sánh với các ngưỡng để xác định hướng:
 - $0^\circ \pm 15^\circ$ hoặc $345^\circ - 360^\circ$: "rẽ phải"
 - $180^\circ \pm 15^\circ$: "rẽ trái"
 - $90^\circ \pm 15^\circ$ hoặc $270^\circ \pm 15^\circ$: "đi thẳng"
 - Góc chéo: ưu tiên trái/phải nhưng confidence thấp hơn

- Tính confidence dựa trên tỷ lệ đỉnh histogram và mức độ chiếm ưu thế của hướng đó.
- Region-based Analysis
 - Khi phát hiện điểm giao nhau (ngã ba/ngã tư), chia ảnh thành các vùng (trên, dưới, trái, phải).

```
def _region_based_analysis(self, gray: np.ndarray, junction_info: dict) -> tuple:
    """Phân tích theo vùng để xác định hướng cho junction"""
    try:
        height, width = gray.shape
        directions = {}

        # Chia ảnh thành 4 vùng: trên, dưới, trái, phải
        regions = {
            'top': gray[:height//2, :],
            'bottom': gray[height//2:, :],
            'left': gray[:, :width//2],
            'right': gray[:, width//2:]
        }
    
```

- Phân tích mật độ biên và mẫu mũi tên trong từng vùng để xác định hướng nào có mũi tên.

```
# Phân tích density và edge count cho mỗi vùng
region_scores = {}
for region_name, region in regions.items():
    edges = cv2.Canny(region, 50, 150)
    edge_density = np.sum(edges) / (region.shape[0] * region.shape[1])

    # Detect arrow patterns trong region
    arrow_score = self._detect_arrow_in_region(region)

    combined_score = edge_density * 0.6 + arrow_score * 0.4
    region_scores[region_name] = combined_score

# Map regions to directions
region_to_direction = {
    'top': 'đi thẳng',      # Mũi tên hướng lên
    'bottom': 'đi thẳng',   # Mũi tên hướng xuống
    'left': ' rẽ trái',     # Mũi tên hướng trái
    'right': ' rẽ phải'     # Mũi tên hướng phải
}
```



```

# Lọc directions có score cao
threshold = np.mean(list(region_scores.values())) + np.std(list(region_scores.values())) * 0.5

for region_name, score in region_scores.items():
    if score > threshold:
        direction = region_to_direction[region_name]
        if direction not in directions:
            directions[direction] = 0.0
        directions[direction] += score

# Normalize scores
total_score = sum(directions.values())
if total_score > 0:
    for direction in directions:
        directions[direction] /= total_score

overall_confidence = min(total_score * 0.8, 1.0) # Scale down confidence

return directions, overall_confidence

except Exception as e:
    print(f"[ERROR] Region-based analysis: {e}")
    return {}, 0.0

```

c. Tổng hợp kết quả

- Kết quả từ các phương pháp trên được kết hợp bằng voting có trọng số (weighted voting).

```

793 def _combine_multi_results(self, tm_directions, contour_directions, edge_direction, tm_conf, contour_conf, edge_conf) -> tuple:
794     """Combine kết quả từ nhiều phương pháp với multi-direction support được cải thiện"""
795     all_directions = {}
796
797     # Cải thiện weight distribution dựa trên reliability
798     tm_weight = 0.45 if tm_conf > 0.4 else 0.35
799     contour_weight = 0.35 if contour_conf > 0.5 else 0.25
800     edge_weight = 0.20 if edge_conf > 0.3 else 0.15
801
802     # Merge template matching results với weight cải thiện
803     for direction, confidence in tm_directions.items():
804         if direction not in all_directions:
805             all_directions[direction] = 0.0
806         # Áp dụng penalty cho confidence quá thấp
807         adjusted_conf = confidence if confidence >= 0.25 else confidence * 0.5
808         all_directions[direction] += adjusted_conf * tm_weight * tm_conf
809
810     # Merge contour analysis results với validation tốt hơn
811     for direction, confidence in contour_directions.items():
812         if direction not in all_directions:
813             all_directions[direction] = 0.0
814         # Bonus cho contour với confidence cao
815         adjusted_conf = confidence * 1.1 if confidence >= 0.7 else confidence
816         all_directions[direction] += adjusted_conf * contour_weight * contour_conf
817
818     # Add edge analysis result với conditional weighting
819     if edge_direction != "unknown" and edge_conf > 0.15:
820         if edge_direction not in all_directions:
821             all_directions[edge_direction] = 0.0
822         # Edge analysis weight dựa trên edge confidence
823         dynamic_edge_weight = min(edge_weight * (edge_conf / 0.5), 0.3)
824         all_directions[edge_direction] += edge_conf * dynamic_edge_weight

```

```

825
826 # Normalize và filter với threshold adaptive
827 total_score = sum(all_directions.values())
828 if total_score > 0:
829     normalized_directions = {}
830     max_score = max(all_directions.values())
831
832     for direction, score in all_directions.items():
833         normalized_score = score / total_score
834         # Adaptive threshold: cao hơn nếu có 1 direction dominant
835         threshold = 0.15 if max_score / total_score > 0.6 else 0.12
836
837         if normalized_score > threshold:
838             # Boost confidence cho direction có score cao rõ rệt
839             if normalized_score > 0.4:
840                 normalized_score = min(normalized_score * 1.1, 1.0)
841             normalized_directions[direction] = normalized_score
842
843     # Re-normalize after boosting
844     final_total = sum(normalized_directions.values())
845     if final_total > 0:
846         for direction in normalized_directions:
847             normalized_directions[direction] /= final_total
848
849     final_confidence = min(total_score * 1.2, 1.0) # Slight boost cho overall confidence
850     return normalized_directions, final_confidence
851
852 return {"unknown": 1.0}, 0.0
853

```

- Nếu nhiều phương pháp đồng thuận về một hướng, confidence sẽ được tăng lên.

```

1020
1021 def _combine_results(self, results) -> tuple:
1022     """Combine kết quả từ nhiều phương pháp với weighted voting cải thiện"""
1023     direction_scores = {}
1024     total_confidence = 0.0
1025
1026     for direction, confidence, weight in results:
1027         if direction != "unknown" and confidence > 0.1: # Chỉ xét confidence > threshold
1028             if direction not in direction_scores:
1029                 direction_scores[direction] = 0.0
1030             weighted_score = confidence * weight
1031             direction_scores[direction] += weighted_score
1032             total_confidence += weighted_score
1033
1034     if not direction_scores:
1035         return "unknown", 0.0
1036
1037     # Chọn direction có score cao nhất
1038     best_direction = max(direction_scores, key=direction_scores.get)
1039     best_score = direction_scores[best_direction]
1040
1041     # Normalize score
1042     if total_confidence > 0:
1043         normalized_score = best_score / total_confidence
1044     else:
1045         normalized_score = 0.0
1046
1047     # Kiểm tra consensus - nếu 2 methods đồng ý thì tăng confidence
1048     consensus_count = sum(1 for direction, conf, weight in results if direction == best_direction and conf > 0.2)
1049     if consensus_count >= 2:
1050         normalized_score *= 1.2 # Bonus cho consensus
1051         print(f"[CV DEBUG] Consensus bonus: {consensus_count} methods agree on {best_direction}")
1052
1053     return best_direction, min(normalized_score, 1.0) # Cap tại 1.0
1054

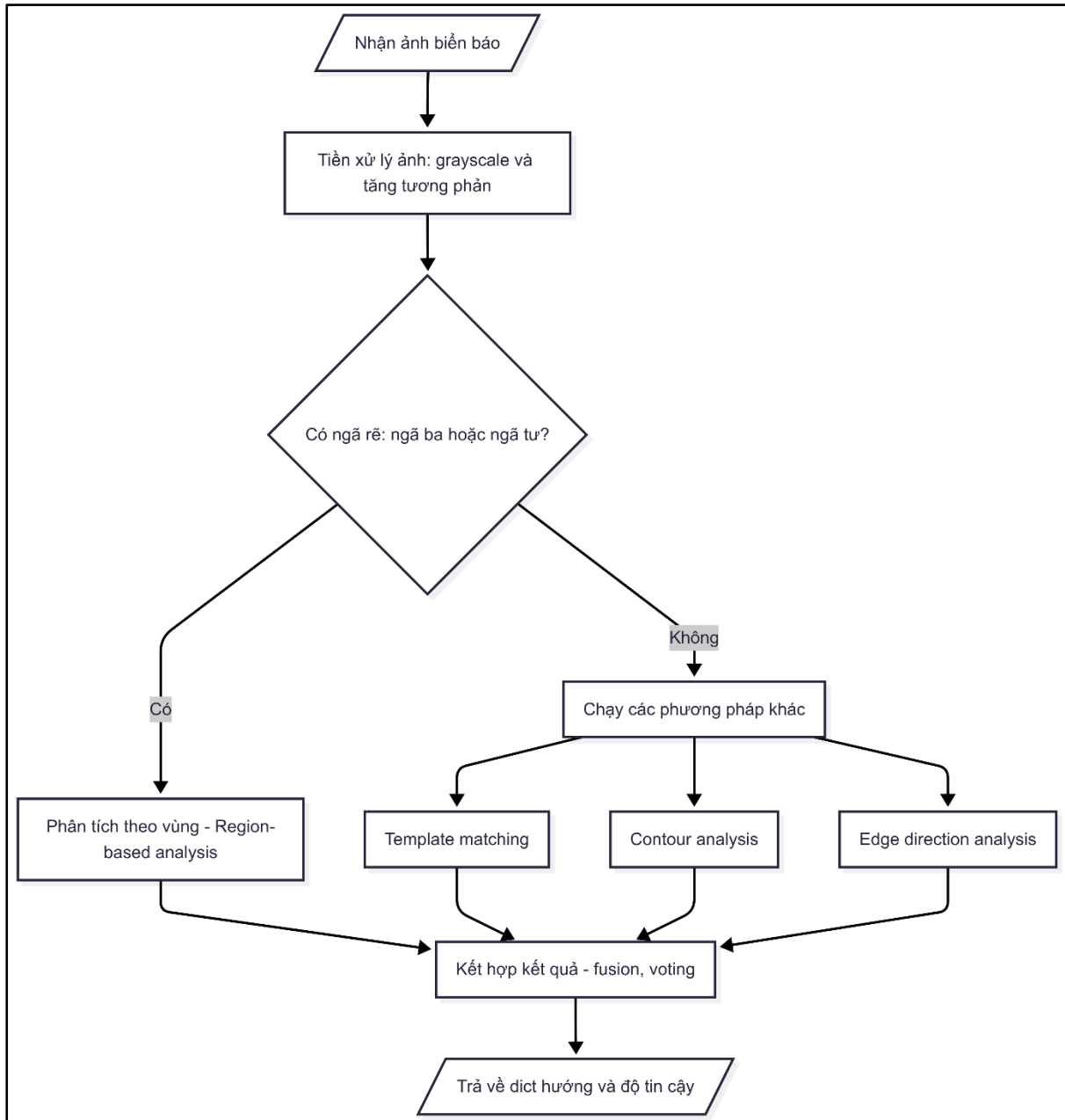
```

- Hỗ trợ nhận diện nhiều hướng cùng lúc (ví dụ: biển báo chỉ dẫn nhiều hướng tại ngã ba/ngã tư): Hàm `_combine_multi_results` trả về một dict chứa nhiều hướng và confidence cho từng hướng, ví dụ:

```
{ 'rẽ trái': 0.55, 'đi thẳng': 0.45 }
```

d. Quy trình tổng quát

- Tiền xử lý ảnh (grayscale, tăng tương phản).
- Kiểm tra có phải điểm giao nhau các ngã rẽ không (nếu có, ưu tiên region-based analysis).
- Chạy các phương pháp: template matching, contour analysis, edge direction analysis.
- Kết hợp kết quả, trả về dictionary các hướng và độ tin cậy tổng thể.



< Workflow tổng quát khi xác định được biển báo có mũi tên >

2.3.4.c. Đọc ký tự trên biển báo

a. Crop vùng biển báo nghi ngờ có chữ.

Hệ thống sẽ xác định và cắt (crop) vùng ảnh chứa biển báo giao thông từ khung hình gốc dựa trên kết quả xác định Object Detection từ mô hình YOLOv8.

b. Đưa ảnh vào mô hình Vintern để sinh ra text mô tả ký tự trên biển báo.

- Ảnh crop được chuyển sang định dạng phù hợp (PIL, resize, chuyển màu).
- Đưa ảnh vào mô hình Vintern NLP (`self.vintern_model.chat`) để sinh ra text mô tả nội dung được ghi trên biển báo.

```
pil_image = Image.fromarray(cv2.cvtColor(sign_image, cv2.COLOR_BGR2RGB))
pixel_values = load_image_for_vintern(pil_image, input_size=448, max_num=4).to(torch.float16).to(self.device)
```

c. Tiền xử lý text để loại bỏ thông tin không cần thiết.

```
def _get_text_from_sign(self, sign_image: np.ndarray) -> str:
    """Hybrid method: Computer Vision + NLP"""
    if self.vintern_model is None or self.vintern_tokenizer is None:
        return ""
    try:
        # BƯỚC 1: Computer Vision Arrow Detection (Multi-Direction)
        cv_directions, cv_confidence = self.arrow_detector.detect_arrow_direction(sign_image)

        # BƯỚC 2: NLP Text Reading
        pil_image = Image.fromarray(cv2.cvtColor(sign_image, cv2.COLOR_BGR2RGB))
        pixel_values = load_image_for_vintern(pil_image, input_size=448, max_num=4).to(torch.float16).to(self.device)
        generation_config = dict(max_new_tokens=200, do_sample=False, num_beams=3, repetition_penalty=1.3, pad_token_id=self.vintern_tokenizer.eos_token_id)

        # Prompt cực kỳ đơn giản
        question = """<image> Đọc text"""

        response, history = self.vintern_model.chat(self.vintern_tokenizer, pixel_values, question, generation_config, history=None, return_history=True)

        # Xử lý NLP response
        response = response.strip()
        response = response.replace('***', '').replace(' ', '').replace('*', '')

        # Tách các dòng và lọc thông tin hữu ích
        all_lines = []
        for line in response.split('\n'):
            line = line.strip()
            if line and len(line) >= 3 and len(line) <= 120:
                # Loại bỏ các dòng mô tả không cần thiết
                if not any(skip in line.lower() for skip in [
                    'biển báo', 'hình ảnh', 'theo thứ tự', 'liệt kê', 'đọc được',
                    'nhìn thấy', 'có thể thấy', 'xuất hiện', 'tôi thấy', 'dựa vào'
                ]):
                    # Làm sạch dòng
                    clean_line = line.lstrip('- ').rstrip('* ').rstrip('1234567890. ').strip()
                    if clean_line and len(clean_line) >= 2:
                        all_lines.append(clean_line)

        nlp_text = ""
        if all_lines:
            nlp_text = ' | '.join(all_lines[:4]) # Lấy tối đa 4 dòng đầu tiên
        else:
            # Fallback: lấy response gốc và làm sạch
            fallback = response.replace('\n', ' ').strip()
            if len(fallback) > 200:
                fallback = fallback[:200] + "..."
            nlp_text = fallback

        # BƯỚC 3: Fusion CV + NLP Results (Multi-Direction)
        final_result = self.fuse_cv_nlp_results_multi(cv_directions, cv_confidence, nlp_text)
        print(f"[FINAL RESULT] {final_result}")

        return final_result

    except Exception as e:
        print(f"[ERROR] Lỗi khi xử lý Hybrid CV+NLP: {e}")
        return ""
```

2.3.4.d. Kết hợp Computer Vision (Arrow Detector) với NLP

Mô hình xử lý ngôn ngữ tự nhiên được sử dụng là **Vintern-1B v3.5**. Đây là mô hình ngôn ngữ lớn hỗ trợ rất tốt Tiếng Việt, đồng thời có ưu điểm là cực kỳ nhẹ. Hệ thống sẽ sử dụng mô hình này để nhận diện và trích xuất ký tự Tiếng Việt được ghi trên biển báo giao thông, từ đó kết hợp với thuật toán xác định hướng mũi tên (Arrow Detector) để đưa ra kết quả chính xác về hướng đi, địa điểm và khoảng cách cho người dùng.

a. Phân tích ký tự

- Sử dụng hàm `_parse_locations_from_nlp` để trích xuất các địa điểm và khoảng cách từ text sinh ra bởi Vintern.
- Hỗ trợ nhiều pattern như: "Chợ Thái 3.3 Km", "3,3 Km Chợ Thái",...

b. Kết với kết quả từ Arrow Detector với thông tin từ NLP

- Nếu có nhiều hướng từ CV (multi-direction) và nhiều địa điểm từ NLP, sẽ map từng hướng với từng địa điểm theo độ tin cậy (confidence) và logic ưu tiên. Và trả về kết quả theo định dạng:

"rẽ trái 3km đến Chợ Thái | đi thẳng 50km đến Hà Nội"

- Nếu chỉ có một phía (CV hoặc NLP) đủ thông tin, sẽ ưu tiên phía đó.
- Nếu cả hai phía đều không đủ thông tin, trả về thông báo chưa xác định.

2.3.4.e. Quản lý Cache OCR

a. Mục đích

Tránh nhận diện lại OCR (Optical Character Recognition - công nghệ nhận diện và chuyển đổi ký tự từ hình ảnh sang dạng văn bản số) các biển báo đã xử lý ở các frame trước, giúp tăng tốc và giảm tài nguyên sử dụng cho hệ thống.

b. Cách hoạt động

- Mỗi object (biển báo) được lưu cache kết quả OCR theo `object_id`.
- Kiểm tra cache dựa trên:
 - Hash ảnh: Nếu nội dung ảnh crop không đổi.
 - Vị trí bbox: Nếu bounding box không thay đổi nhiều (dùng IoU).
 - Thời gian: Nếu cache chưa hết hạn (timeout).
- Nếu cache hợp lệ thì dùng lại kết quả, nếu không thì xóa cache và nhận diện lại.

```

207
208 def _is_cache_valid(self, object_id: int, current_bbox: list, image_hash: str) -> bool:
209     """Kiểm tra cache có còn hợp lệ không"""
210     current_time = time.time()
211
212     # Kiểm tra timeout
213     if object_id in self.ocr_timestamp_cache:
214         if current_time - self.ocr_timestamp_cache[object_id] > self.ocr_cache_timeout:
215             return False
216
217     # Kiểm tra bbox thay đổi
218     if object_id in self.ocr_bbox_cache:
219         old_bbox = self.ocr_bbox_cache[object_id]
220         iou_bbox = bbox_iou(current_bbox, old_bbox)
221         if iou_bbox < 0.7:
222             return False
223
224     # Kiểm tra nội dung ảnh thay đổi (quan trọng nhất)
225     if object_id in self.ocr_image_hash_cache:
226         old_hash = self.ocr_image_hash_cache[object_id]
227         if old_hash != image_hash:
228             return False
229
230     return True
231

```

```

1791
1792     # Tính hash của ảnh crop để xác định nội dung
1793     image_hash = self._compute_image_hash(sign_crop)
1794     if not image_hash:
1795         detection['ocr_text'] = "Hash error"
1796         detections.append(detection)
1797         continue
1798
1799     # Kiểm tra cache có còn hợp lệ không
1800     cache_valid = self._is_cache_valid(object_id, current_bbox, image_hash)
1801
1802     if cache_valid and object_id in self.ocr_cache:
1803         # Sử dụng cache hợp lệ
1804         detection['ocr_text'] = self.ocr_cache[object_id]
1805     else:
1806         # Cache không hợp lệ hoặc không tồn tại
1807         if not cache_valid:
1808             self._clear_object_cache(object_id)
1809
1810         # Cập nhật cache với thông tin mới
1811         self._update_object_cache(object_id, current_bbox, image_hash)
1812
1813         # Gửi tới OCR queue nếu chưa gửi
1814         if (object_id, ) not in self.ocr_queue_sent:
1815             self.ocr_queue.put((object_id, sign_crop.copy()))
1816             self.ocr_queue_sent.add((object_id, ))
1817
1818         detection['ocr_text'] = "⌚ Hybrid Processing..."
1819         detections.append(detection)
1820     return detections
1821

```

2.3.4.f. Theo dõi đối tượng (Object Tracking)

a. Mục đích

Gán `object_id` cố định, duy nhất cho từng biển báo và duy trì qua các frame liên tiếp giúp quản lý cache và theo dõi chuyển động của đối tượng.

b. Cách thực hiện

- Sau khi YOLO phát hiện các bounding box, các box này được đưa vào thuật toán ***SORT (Simple Online and Realtime Tracking)***.
- ***SORT*** sử dụng *Kalman Filter* và *Hungarian Algorithm* để gán ID cho từng object, đảm bảo mỗi object giữ nguyên ID khi di chuyển qua các frame (nếu không bị mất track).
- Kết hợp với cache OCR
 - Mục đích: Đảm bảo mỗi object chỉ nhận diện OCR một lần khi không thay đổi về vị trí, hình ảnh, thời gian.
 - Cách hoạt động:
 - Mỗi object được lưu cache OCR dựa trên `object_id`
 - Khi object xuất hiện ở frame mới, hệ thống sẽ kiểm tra:
 - Hash ảnh: Nếu hash ảnh crop từ bbox giống với lần trước.
 - BBox IoU: Nếu vị trí bbox không thay đổi nhiều (IoU đủ lớn).
 - Timeout: Nếu chưa quá thời gian cho phép.
 - Nếu cache còn hợp lệ, lấy kết quả OCR từ cache, không cần nhận diện lại.
 - Nếu cache không hợp lệ (object di chuyển, thay đổi hoặc hết hạn) thì xóa cache cũ, cập nhật cache mới và gửi object vào hàng đợi OCR.

```

1798
1799         # Kiểm tra cache có còn hợp lệ không
1800         cache_valid = self._is_cache_valid(object_id, current_bbox, image_hash)
1801
1802         if cache_valid and object_id in self.ocr_cache:
1803             # Sử dụng cache hợp lệ
1804             detection['ocr_text'] = self.ocr_cache[object_id]
1805         else:
1806             # Cache không hợp lệ hoặc không tồn tại
1807             if not cache_valid:
1808                 self._clear_object_cache(object_id)
1809
1810             # Cập nhật cache với thông tin mới
1811             self._update_object_cache(object_id, current_bbox, image_hash)
1812
1813             # Gửi tới OCR queue nếu chưa gửi
1814             if (object_id, ) not in self.ocr_queue_sent:
1815                 self.ocr_queue.put((object_id, sign_crop.copy()))
1816                 self.ocr_queue_sent.add((object_id, ))
1817
1818             detection['ocr_text'] = "⌚ Hybrid Processing..."
1819         detections.append(detection)

```

2.3.4.g. Hiển thị kết quả

Hiển thị kết quả nhận diện lên ảnh hoặc video, bao gồm Class ID (nhãn), tên biển báo được ánh xạ từ Class ID, độ tin cậy, kết quả OCR.

```

1821
1822     def draw_and_show(self, frame: np.ndarray, detections):
1823         # Vẽ bounding box bằng OpenCV trước
1824         for det in detections:
1825             x1, y1, x2, y2 = map(int, det['bbox'])
1826             color = (0, 255, 0)
1827             cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
1828         # Chuyển frame sang PIL để vẽ text Unicode
1829         img_pil = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
1830         draw = ImageDraw.Draw(img_pil)
1831         font_path = os.path.join(os.path.dirname(__file__), "arial.ttf")
1832         if not os.path.exists(font_path):
1833             font_path = "C:/Windows/Fonts/arial.ttf"
1834         try:
1835             font = ImageFont.truetype(font_path, 20)
1836         except:
1837             font = ImageFont.load_default()
1838         for det in detections:
1839             x1, y1, x2, y2 = map(int, det['bbox'])
1840             # Lấy mã biển báo từ class_names thay vì dùng class_id_code hardcoded
1841             class_code = det['class_label'] if 'class_label' in det else f"Class_{det['class_id']}"
1842             description_no_diacritics = det['class_label_vi'] if 'class_label_vi' in det else class_code
1843             confidence = det['confidence']
1844
1845             # Format: Mã biển | Description_tiếng_viet_khong_dai | Độ tin cậy
1846             label = f"{class_code} | {description_no_diacritics} | {confidence:.1%}"
1847             draw.text((x1, y1 - 25), label, font=font, fill=(0,255,0))
1848
1849             # Chỉ hiển thị OCR khi có kết quả thành công (không phải "... hay "⌚ Processing...")
1850             if (det.get('ocr_text') and
1851                 det['ocr_text'] != "..." and
1852                 not det['ocr_text'].startswith("⌚ ") and
1853                 not det['ocr_text'].startswith("Invalid") and
1854                 not det['ocr_text'].startswith("Hash error") and
1855                 len(det['ocr_text'].strip()) > 0):
1856                 ocr_label = f"HYBRID: {det['ocr_text']}"
1857                 draw.text((x1, y2 + 5), ocr_label, font=font, fill=(255,165,0)) # Orange cho hybrid
1858                 print(f"{class_code} | {description_no_diacritics} | HYBRID: {det['ocr_text']}")
1859         frame_show = cv2.cvtColor(np.array(img_pil), cv2.COLOR_RGB2BGR)
1860         cv2.imshow('Traffic Sign Detection - Hybrid CV+NLP', frame_show)
1861

```


2.3.5. Luồng xử lý chính

1. Đọc frame từ webcam/video

- Hàm `run_webcam` hoặc `run_video` sử dụng OpenCV (`cv2.VideoCapture`) để lấy từng frame từ camera hoặc file video.
- Mỗi frame được truyền vào hàm `predict_frame` để xử lý.

2. Phát hiện biển báo bằng YOLOv8

- Trong `predict_frame`, frame được tăng cường chất lượng (nếu bật) rồi đưa vào model YOLOv8 (`self.model.predict`).
- YOLO trả về danh sách các bounding box, class, confidence cho từng đối tượng phát hiện.

3. Với mỗi biển báo nghi ngờ:

- Lặp qua từng object đã detect, lấy bbox, class, confidence.
- Nếu là biển chỉ dẫn (ví dụ: `class_label == "sus"`), thực hiện các bước tiếp theo.

4. Nếu là biển chỉ dẫn (có thể có mũi tên/chữ), crop vùng ảnh

- Crop vùng ảnh quanh bbox, có padding nhỏ để tránh cắt mất thông tin.
- Kiểm tra crop hợp lệ (không bị out-of-bounds).

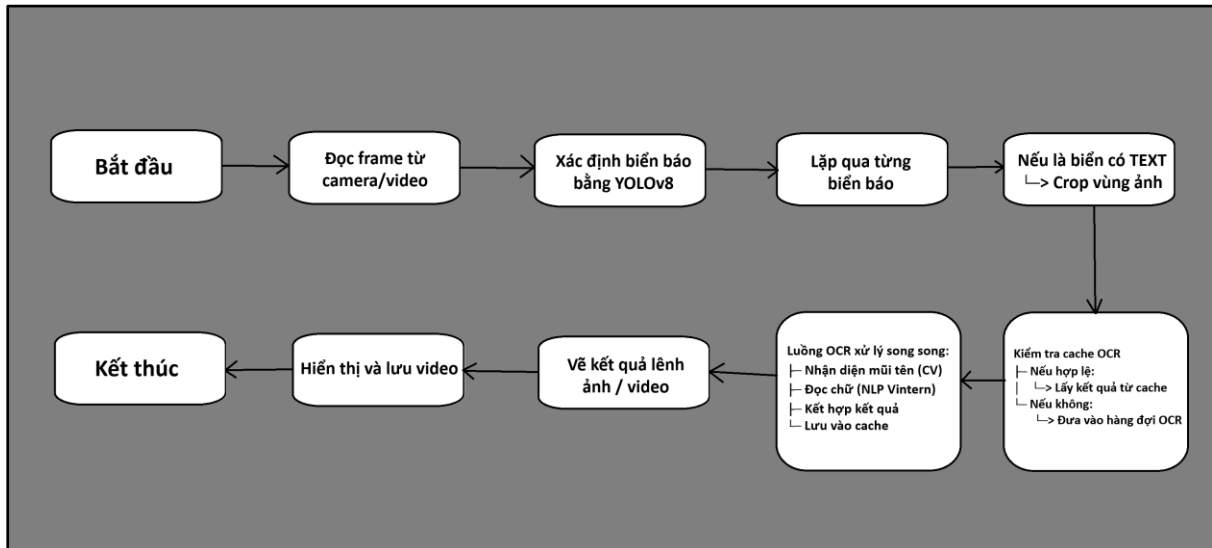
5. Kiểm tra cache, nếu chưa có thì đưa vào hàng đợi OCR

- Tính hash của ảnh crop để xác định nội dung duy nhất.
- Gọi `_is_cache_valid` để kiểm tra cache dựa trên:
 - Thời gian hết hạn (timeout)
 - Bbox thay đổi (IoU)
 - Hash ảnh thay đổi
- Nếu cache hợp lệ và đã có kết quả OCR, lấy luôn từ cache.
- Nếu cache không hợp lệ hoặc chưa có, xóa cache cũ, cập nhật cache mới, và đưa object vào hàng đợi OCR (`self.ocr_queue`).

6. Trong thread OCR:

- Thread OCR (`ocr_worker`) lấy `object_id` và ảnh crop từ queue.
- Tính lại hash để đảm bảo tính nhất quán.
- Gọi `_get_text_from_sign` để xử lý hybrid:
- Nhận diện hướng mũi tên bằng các thuật toán CV: Gọi `self.arrow_detector.detect_arrow_direction`.

- Đọc chữ bằng Vintern NLP: Chuyển ảnh sang PIL, chuẩn hóa, rồi dùng Vintern model để sinh text.
- Kết hợp kết quả CV + NLP: Gọi `_fuse_cv_nlp_results_multi` để hợp nhất hướng mũi tên và text đọc được.
- Lưu kết quả vào cache (`self.ocr_cache`).



7. Lưu vào cache

Sau khi có kết quả OCR, cập nhật cache cho `object_id` (bbox, hash, timestamp, ocr_text).

8. Vẽ kết quả lên ảnh, hiển thị hoặc lưu video

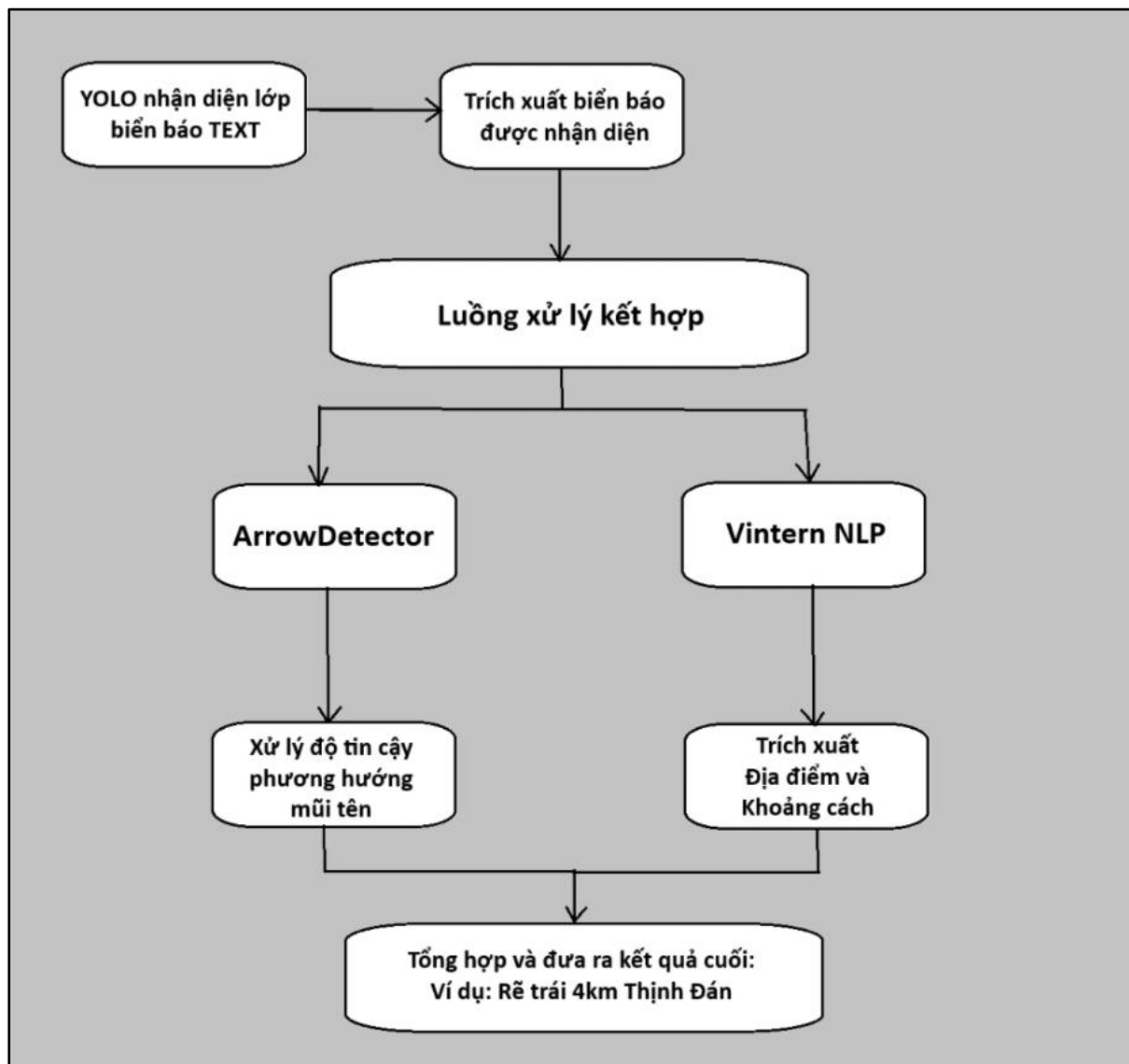
Hàm `draw_and_show`:

- Vẽ bounding box bằng OpenCV.
- Chuyển sang PIL để vẽ text Unicode (label, mô tả, OCR).
- Chỉ hiển thị OCR khi đã có kết quả hợp lệ (không phải "... hoặc " ⌚ Processing...").
- Hiển thị ảnh lên màn hình (`cv2.imshow`) và lưu vào video.

2.3.6. Đặc điểm nổi bật

- Kết hợp đa phương pháp
 - Nhiều thuật toán truyền thống được tích hợp:
 - Template Matching: So khớp mẫu mũi tên với ảnh thực tế.
 - Contour Analysis: Phân tích hình dạng, convex hull, polygon approximation để nhận diện mũi tên.

- Edge Direction Analysis: Sử dụng Sobel, Canny, hoặc các phép toán biên để xác định hướng mũi tên.
 - Morphological Analysis: Dùng các phép toán hình thái để làm nổi bật cấu trúc mũi tên.
 - Junction Detection: Nhận diện các biến giao lộ/phức tạp bằng line detection và intersection analysis.
- Fusion thông minh: Kết quả từ các phương pháp được tổng hợp bằng weighted voting, dynamic threshold, hoặc junction-aware logic để tăng độ tin cậy và giảm lỗi nhận diện.
- Hybrid CV + NLP (Kết hợp thị giác máy tính và xử lý ngôn ngữ tự nhiên)
 - CV nhận diện hướng, NLP đọc chữ: CV xác định hướng mũi tên, NLP (Vintern/EasyOCR) đọc text trên biển báo.
 - Kết hợp thông minh:
 - Mapping hướng ↔ địa điểm: Ghép các hướng với các địa danh dựa trên khoảng cách, logic đường bộ (ví dụ: đường dài = đi thẳng, đường ngắn = rẽ).
 - Xử lý biển nhiều hướng/nhiều địa điểm: Hệ thống có thể xuất ra kết quả theo định dạng “đi thẳng 8km đến TÂN CƯƠNG | rẽ phải 91,5km đến BẮC KẠN”.
 - Sử dụng NLP để xác thực hoặc bổ sung thông tin cho kết quả CV, tăng độ chính xác với biển báo phức tạp.
- Tối ưu hiệu năng
 - Cache thông minh: Lưu kết quả OCR theo object_id, hash ảnh, bbox, timestamp để tránh OCR lặp lại, giảm tải cho model NLP.
 - Thread OCR: Xử lý OCR ở thread riêng, không block luồng chính, đảm bảo tốc độ khung hình cao.
 - Tracking (SORT): Gán object_id ổn định cho từng biển báo, giúp quản lý cache và kết quả mượt mà khi biển di chuyển.
 - Dọn dẹp cache tự động: Xóa cache khi object biến mất hoặc hết hạn, tiết kiệm bộ nhớ.
- Khả năng mở rộng
 - Bổ sung mẫu mũi tên dễ dàng: Chỉ cần thêm template mới hoặc điều chỉnh logic nhận diện trong các hàm template/contour/edge.
 - Cải tiến logic mapping: Có thể nâng cấp thuật toán mapping hướng ↔ địa điểm, thêm rule mới, hoặc tích hợp AI logic.
 - Thay đổi mô hình NLP: Dễ dàng chuyển sang model OCR/NLP khác (Vintern, EasyOCR, TrOCR,...) nhờ thiết kế module hóa.
 - Thêm phương pháp mới: Có thể tích hợp thêm các thuật toán CV hoặc AI mới mà không ảnh hưởng đến kiến trúc tổng thể.



◁ Workflow tổng quát mô tả luồng hệ thống ▷

3. Kết quả

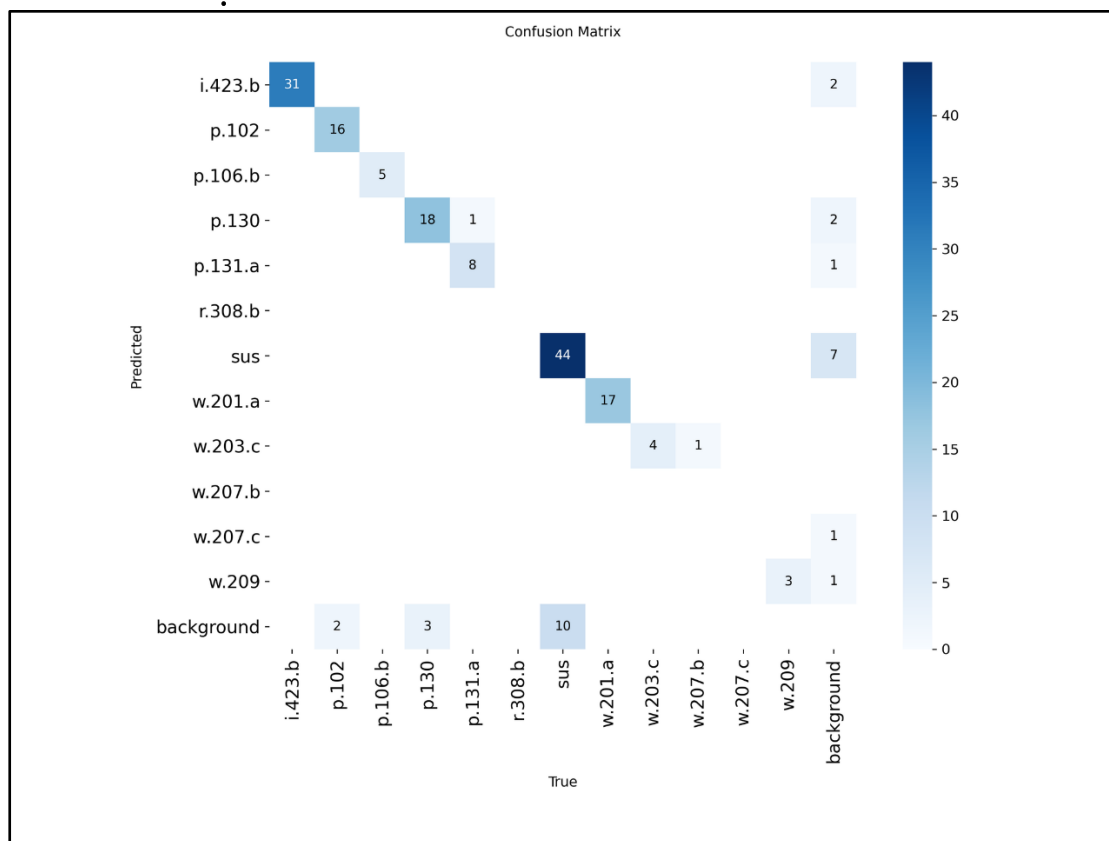
3.1. Số liệu và biểu đồ

Dưới đây là biểu đồ số liệu sau khi huấn luyện thành công mô hình YOLOv8-Medium với bộ dữ liệu thu thập được sau khi được tiến hành tăng cường dữ liệu dưới nhiều góc độ chụp và nhiều điều kiện khác nhau.

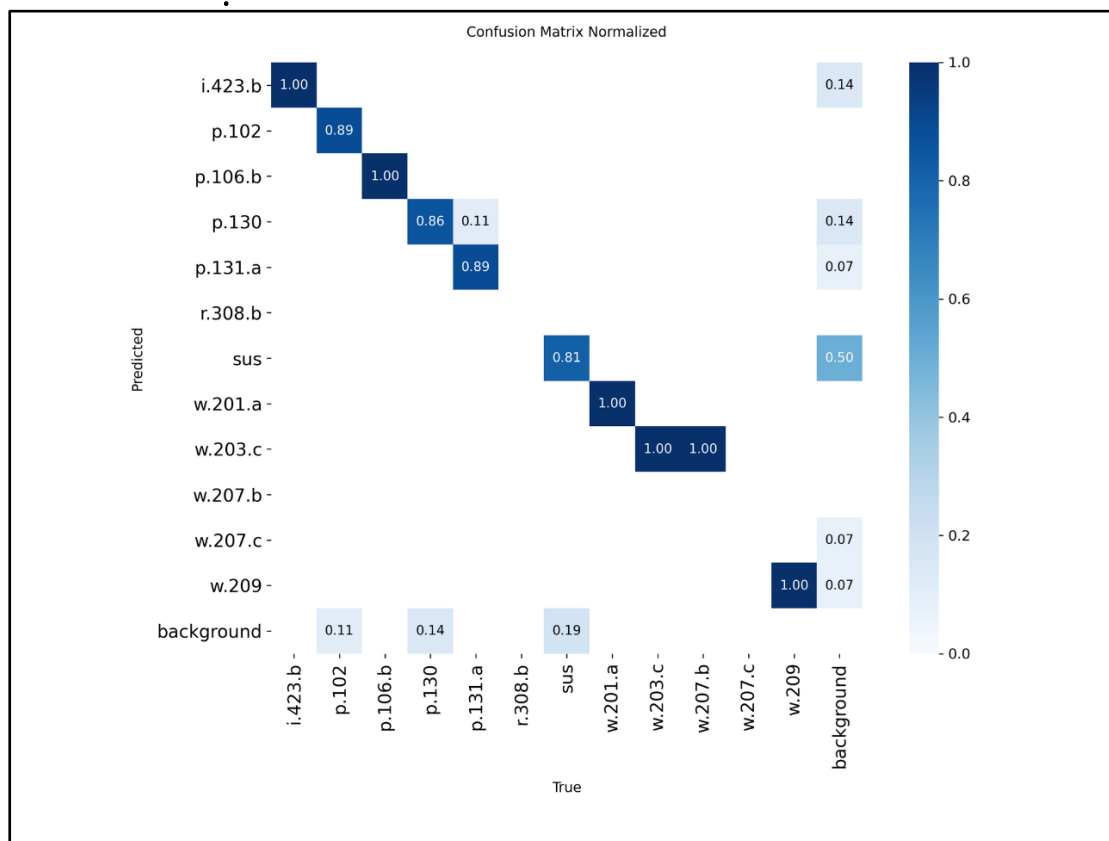
- Tổng số Epoch: 50
- Batch size: 16
- (Thông số được cấu hình chi tiết trong file **config.py**)
- ...

Confusion Matrix	Hiển thị số lượng dự đoán đúng/sai trên từng lớp. → Giúp phân tích lỗi chi tiết.
Confusion Matrix Normalized	Giống ma trận nhầm lẫn nhưng hiển thị tỷ lệ phần trăm. → Giúp dễ so sánh hơn.
Precision-Recall Curve	Thể hiện mối quan hệ giữa độ chính xác (precision) và độ bao phủ (recall) tại nhiều ngưỡng. → Chọn ngưỡng tối ưu.
Recall-Confidence Curve	Thể hiện độ bao phủ (recall) thay đổi thế nào khi thay đổi ngưỡng tin cậy. → Hiểu rõ mức độ bỏ sót.
F1-Confidence Curve	Cho thấy F1-score biến thiên theo confidence, → Chọn ngưỡng tối ưu để cân bằng precision và recall.
Precision-Confidence Curve	Thể hiện độ chính xác thay đổi thế nào khi thay đổi ngưỡng tin cậy. → Giúp đánh giá mức độ nhiễu trong các dự đoán.

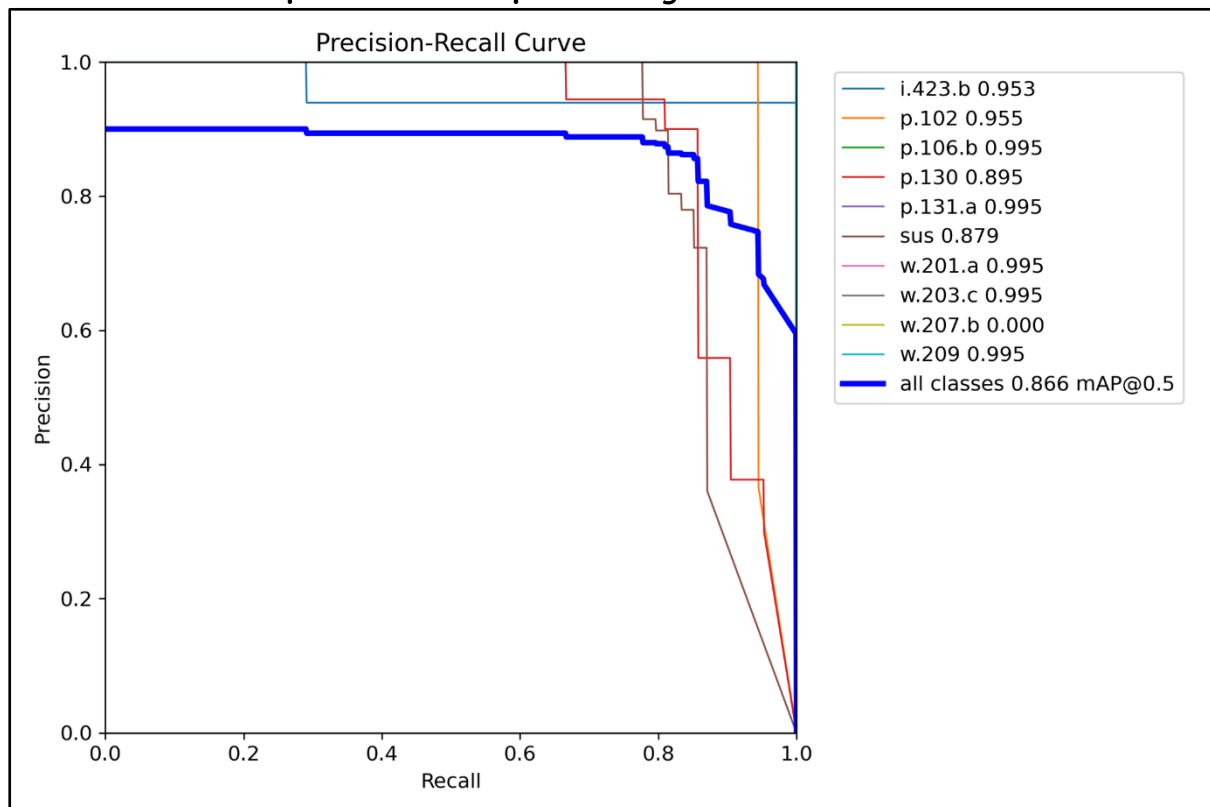
1. Ma trận nhầm lẫn



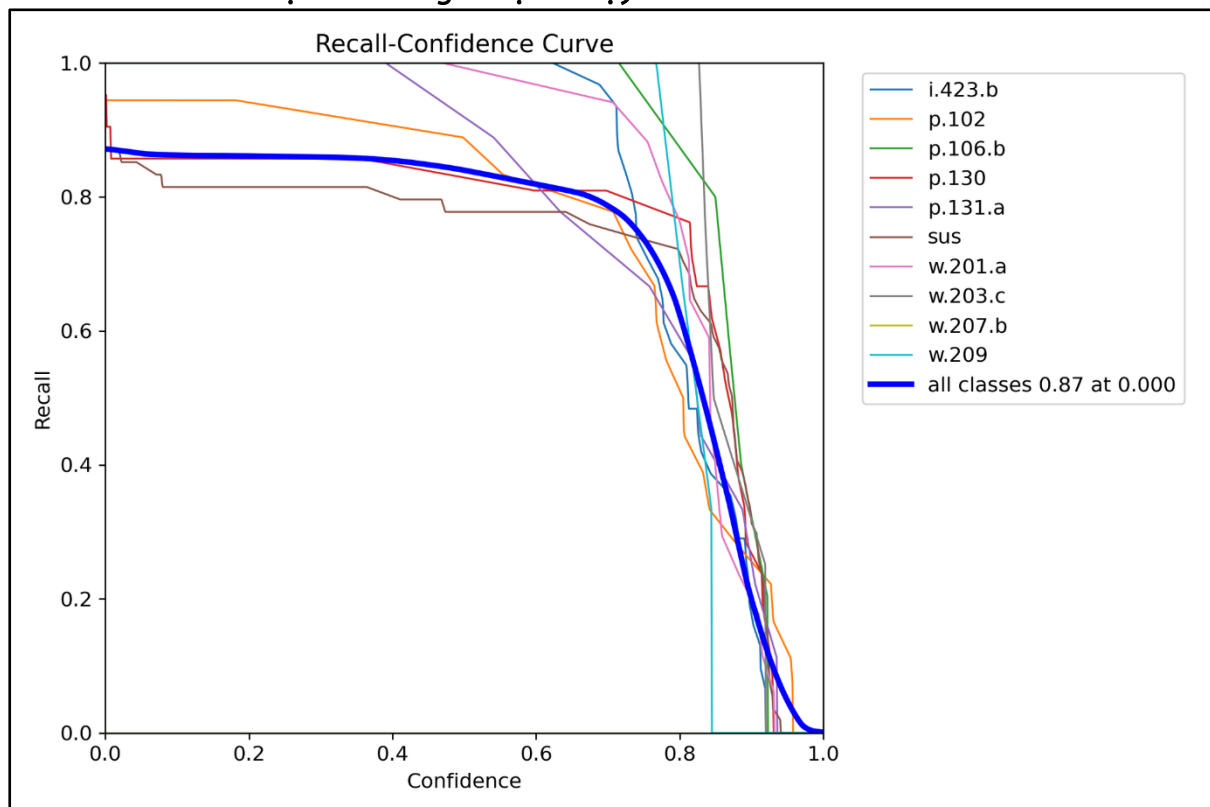
2. Ma trận nhầm lẫn chuẩn hóa



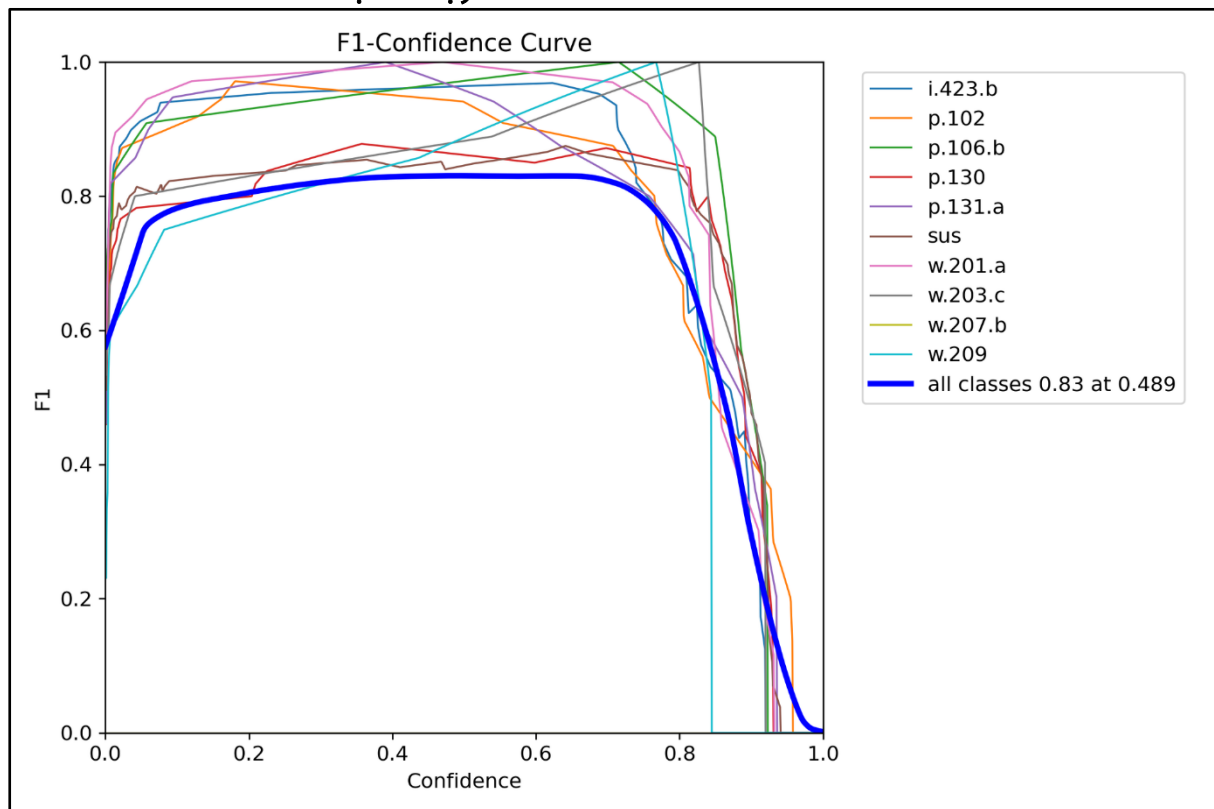
3. Biểu đồ độ chính xác – độ hồi tưởng



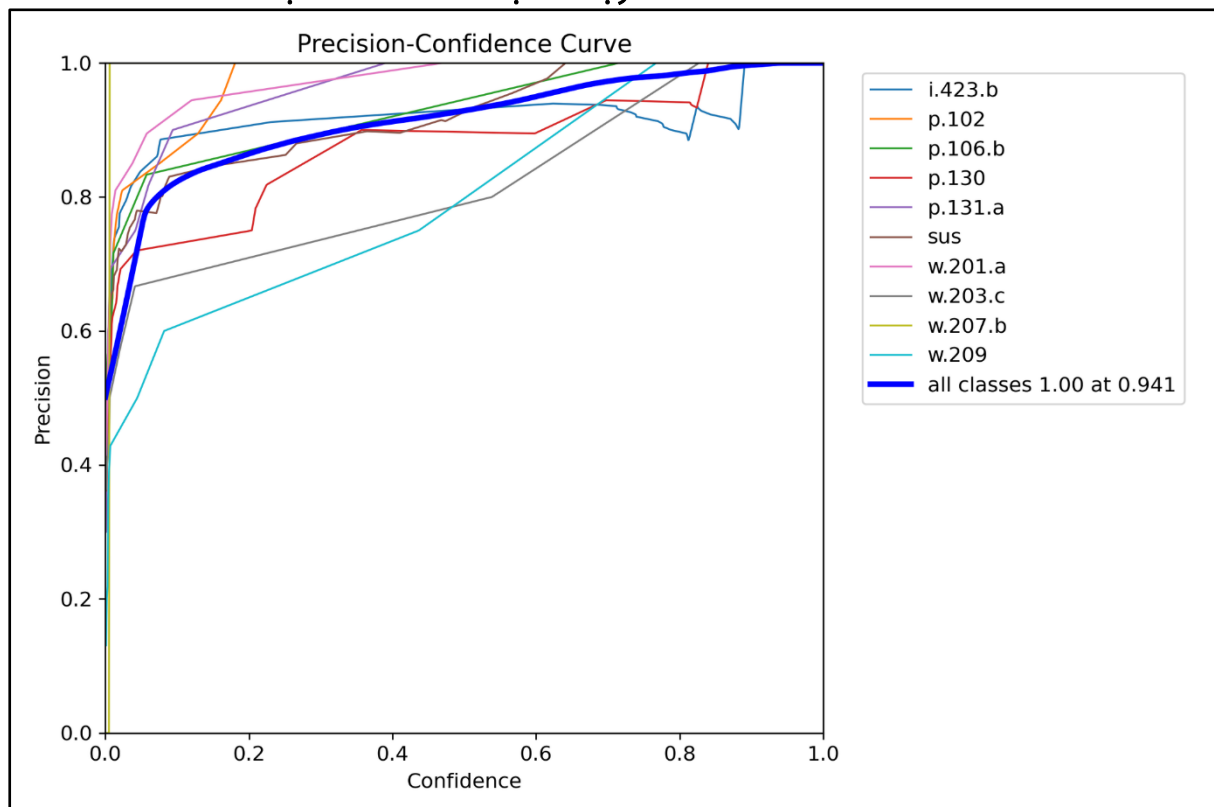
4. Biểu đồ độ hồi tưởng – độ tin cậy



5. Biểu đồ F1 – độ tin cậy



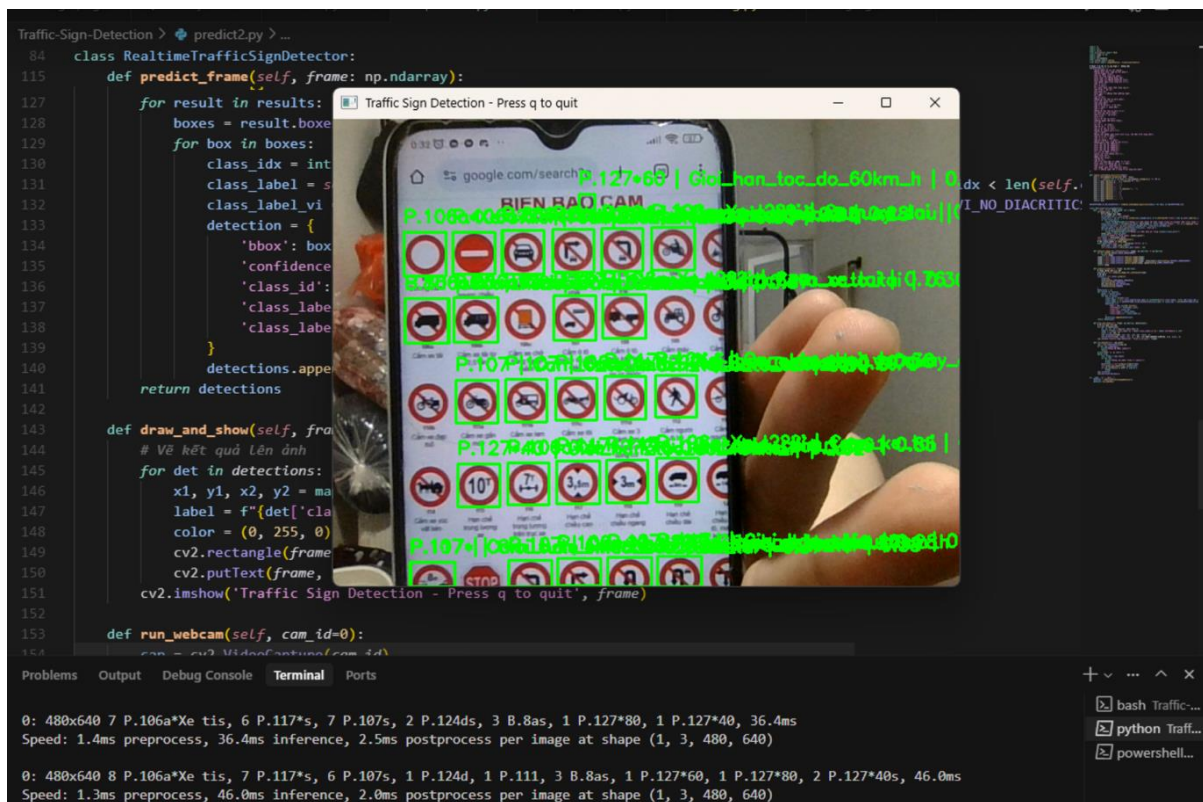
6. Biểu đồ độ chính xác – độ tin cậy



3.2. Thử nghiệm hệ thống

Sau khi xây dựng thành công hệ thống, chúng tôi bước sang công đoạn kiểm thử để xem hệ thống có hoạt động đúng như mong đợi không. Và kết quả mặc dù chưa đạt được như kỳ vọng ban đầu nhưng cũng đủ để đáp ứng kỳ vọng về mặt nghiên cứu của chúng tôi. Hệ thống đủ thông minh để nhận diện chính xác 12 lớp biến báo trong bộ dữ liệu với độ tin cậy khá cao (dao động trong khoảng 60% - 85% trong điều kiện từ trung bình đến tốt). Và đặc biệt hệ thống tích hợp thành công mô hình xử lý ngôn ngữ tự nhiên NLP để phân tích biến báo chứa ký tự.

```
[FINAL RESULT] Không có thông tin phù hợp nào trong ảnh.
[FINAL RESULT] đi thẳng 3.3km đến CHỢ THÁI | rẽ trái 1.5km đến DẦN
[FINAL RESULT] rẽ phải 3.3km đến CHỢ THÁI | rẽ trái 1.6km đến DẦN
[FINAL RESULT] rẽ trái 3.3km đến CHỢ THÁI | đi thẳng 1.6km đến DẦN
[FINAL RESULT] rẽ trái 3.3km đến CHỢ THÁI | đi thẳng 1.6km đến DẦN
[FINAL RESULT] rẽ trái 3.3km đến CHỢ THÁI | đi thẳng 1.6km đến DẦN
[FINAL RESULT] rẽ phải 3.3km đến CHỢ THÁI | đi thẳng 1.6km đến DẦN
[FINAL RESULT] rẽ phải 3.3km đến CHỢ THÁI | đi thẳng 1.6km đến BẮN
[FINAL RESULT] đi thẳng
[FINAL RESULT] rẽ phải 8km đến Tân Cương | đi thẳng 91.5km đến Đắc Kạn
```



4. Kết luận

4.1. Kết luận chung

Về cơ bản dự án đã thành công trong việc huấn luyện và xây dựng một hệ thống nhận diện biển báo giao thông Việt Nam bằng việc ứng dụng mô hình YOLOv8-Medium và tích hợp mô hình xử lý ngôn ngữ tự nhiên Vintern-1B v3.5. Các kết quả thử nghiệm cho thấy mô hình có hiệu năng nhận diện rất tốt các lớp biển báo có trong bộ dữ liệu trong nhiều điều kiện môi trường khác nhau. Việc huấn luyện lại (fine-tuning) mô hình YOLOv8 bằng bộ dữ liệu riêng giúp tăng cường độ chính xác cho việc nhận diện các lớp biển báo đặc thù ở Việt Nam. Ngoài ra việc áp dụng mô hình xử lý ngôn ngữ tự nhiên để xử lý các lớp biển phụ, biển chỉ dẫn có ký tự giúp hệ thống có khả năng nhận diện chi tiết và chính xác các tình huống phức tạp hơn, từ đó góp phần cải thiện độ an toàn, khả năng nhận diện thông tin biển báo, cải thiện trải nghiệm cho người tham gia giao thông tại Việt Nam.

4.2. Hạn chế

Mặc dù đạt được những kết quả khả quan, dự án vẫn còn một số hạn chế nhất định:

- Độ chính xác của mô hình có thể bị ảnh hưởng trong các điều kiện môi trường phức tạp như ánh sáng yếu, mưa to, sương mù, hoặc khi biển báo bị che khuất một phần.
- Số lượng dữ liệu cho một số loại biển báo hiếm gặp vẫn còn hạn chế, dẫn đến khả năng nhận diện chưa tối ưu cho các trường hợp này.
- Ngoài ra, việc triển khai mô hình trên các thiết bị nhúng với tài nguyên hạn chế có thể gặp khó khăn về hiệu năng và tốc độ xử lý do kích thước mô hình.
- Mô hình xử lý ngôn ngữ tự nhiên (NLP) được tích hợp vào dự án chưa thực sự phát huy tối đa công dụng do rào cản về thời gian cũng như kỹ năng của nhóm.
- Dự án mới chỉ dừng ở phạm vi nghiên cứu, chưa mở rộng ra để ứng dụng rộng rãi trong thực tế.

4.3. Tiềm năng phát triển

Để khắc phục các hạn chế và nâng cao hiệu quả của hệ thống, có một số hướng phát triển tiềm năng.

- Thứ nhất, mở rộng và đa dạng hóa bộ dữ liệu bằng cách thu thập thêm hình ảnh trong các điều kiện môi trường khác nhau, cũng như tăng cường số lượng mẫu cho các lớp biến báo ít gặp.
- Thứ hai, nghiên cứu và thử nghiệm các kiến trúc mô hình nhỏ gọn hơn hoặc áp dụng các kỹ thuật tối ưu hóa mô hình (model quantization, pruning) để cải thiện tốc độ và khả năng triển khai trên các thiết bị nhúng.
- Cuối cùng, có thể tích hợp thêm các module xử lý ảnh tiền xử lý nâng cao để cải thiện chất lượng ảnh đầu vào trong điều kiện bất lợi, hoặc kết hợp với các thuật toán theo dõi đối tượng để theo dõi liên tục các biến báo trong chuỗi video.

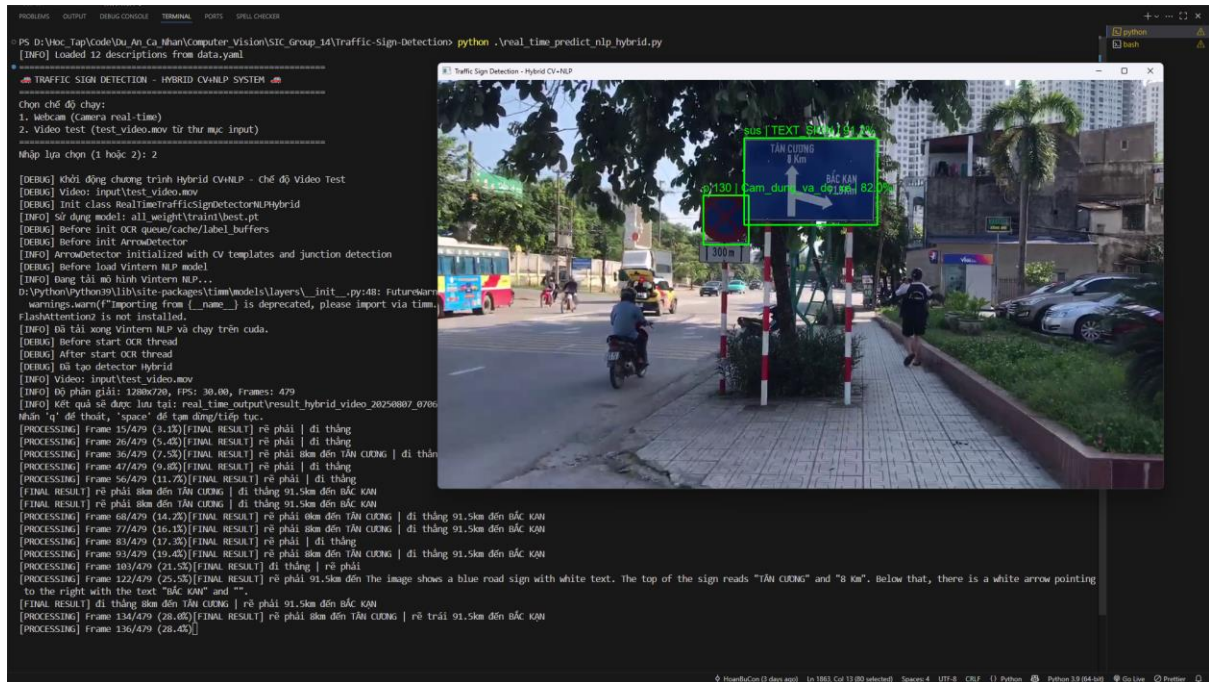
5. Phụ lục và tài liệu tham khảo

5.1. Phụ lục

5.1.1. Biểu đồ hiệu suất mô hình huấn luyện trên Roboflow

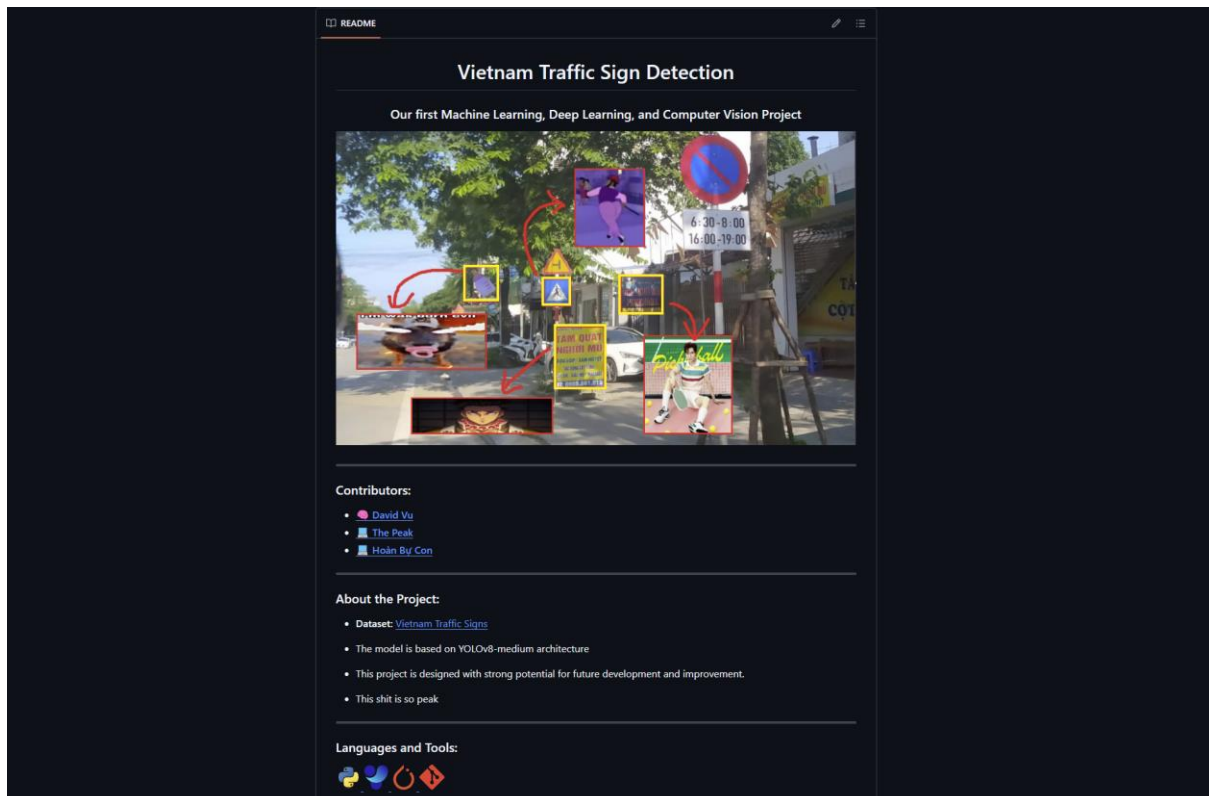


5.1.2. Giao diện Terminal nhận diện biển báo chỉ dẫn có ký tự



5.1.3. Repository GitHub dự án

<https://github.com/HoanBuCon/Traffic-Sign-Detection>



5.2. Tài liệu tham khảo

- Ultralytics – YOLOv8: <https://docs.ultralytics.com/yolov8/>
- GitHub – YOLO: <https://github.com/ultralytics/ultralytics>
- PyTorch: <https://pytorch.org/>
- Mai Tam Dataset – Kaggle: <https://www.kaggle.com/datasets/maitam/vietnamese-traffic-signs>
- Thai Vo Dataset – Kaggle: <https://www.kaggle.com/datasets/thaivothanh/vietnam-traffic-sign>
- ChatGPT, Gemini, Grok3, Claude AI
- Việt Nguyễn AI: <https://www.youtube.com/@vietnh1009>

Mọi thông tin chi tiết xin liên hệ:

- Họ và Tên: Trần Trọng Hoàn
- Email: hoanbucon@gmail.com
- Website: <https://hoanbucon.id.vn>
- Đơn vị: Khoa Công nghệ thông tin – Trường đại học Công nghệ thông tin và Truyền thông - ICTU

Xin chân thành cảm ơn !

6. Nhận xét và đánh giá của thành viên



◁ Ảnh chụp nhóm “Bụi bay vào mắt Quỷ” ▷

TÊN	NHẬN XÉT và ĐÁNH GIÁ

7. Nhận xét và đánh giá của giảng viên

DANH MỤC	ĐIỂM	NHẬN XÉT và ĐÁNH GIÁ
Ý TƯỞNG	__/10	
HỆ THỐNG	__/30	
KẾT QUẢ	__/30	
QUẢN LÝ DỰ ÁN	__/10	
PRESENTATION & REPORT	__/20	
TỔNG CỘNG	__/100	